

Chap.11 강화 학습 (Reinforcement Learning)

방 수 식 교수
(bang@tukorea.ac.kr)

한국공학대학교 전자공학부

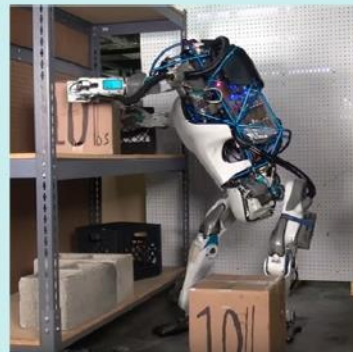
2024년도 2학기
고급머신러닝실습 & 인공지능설계실습2

■ 강화 학습의 과업 사례

- [그림 9-1]의 과업은 상태 변화와 행동을 번갈아 수행하면서 목적을 달성
- 지도 학습이나 비지도 학습으로는 과업 달성 불가능 → 강화 학습으로 해결



(a) 사람의 걷기 학습 과정



(b) 2족 로봇의 걷기

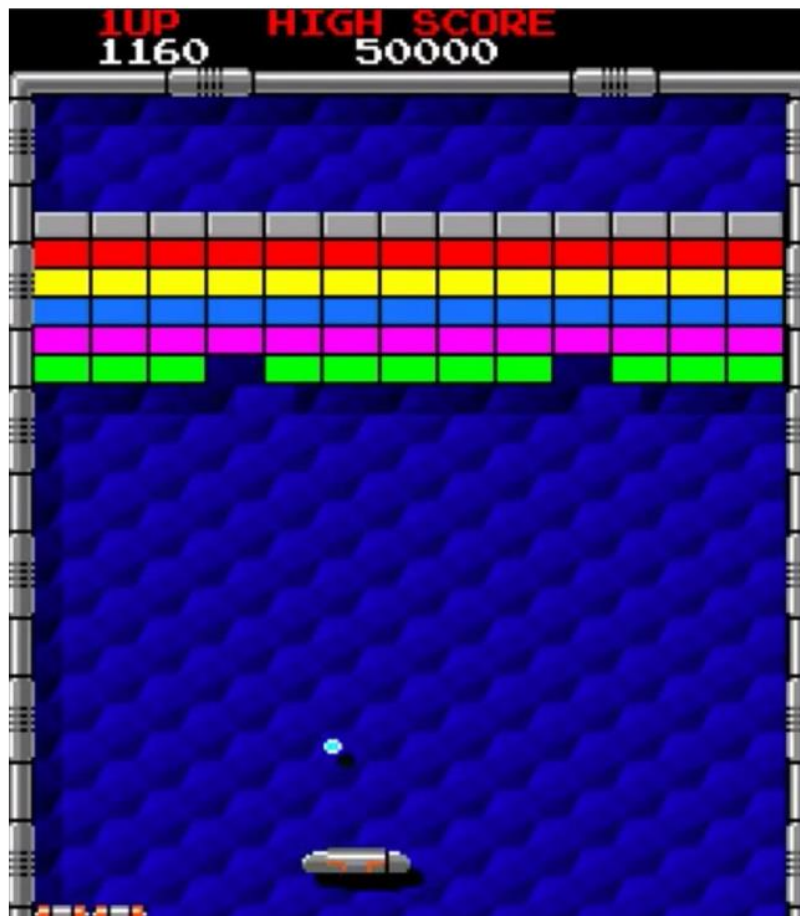
그림 9-1 걷기 과업

Breakout (벽돌깨기 게임)



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

조금 단순하게 접근해 봅시다.



게임의 목표는?

해당 그림만 보았을 때, 최적의 Action은?

고득점을 얻기 위한 전략은?

본 게임에서 취할 수 있는 Action?

Breakout 강화학습 적용



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

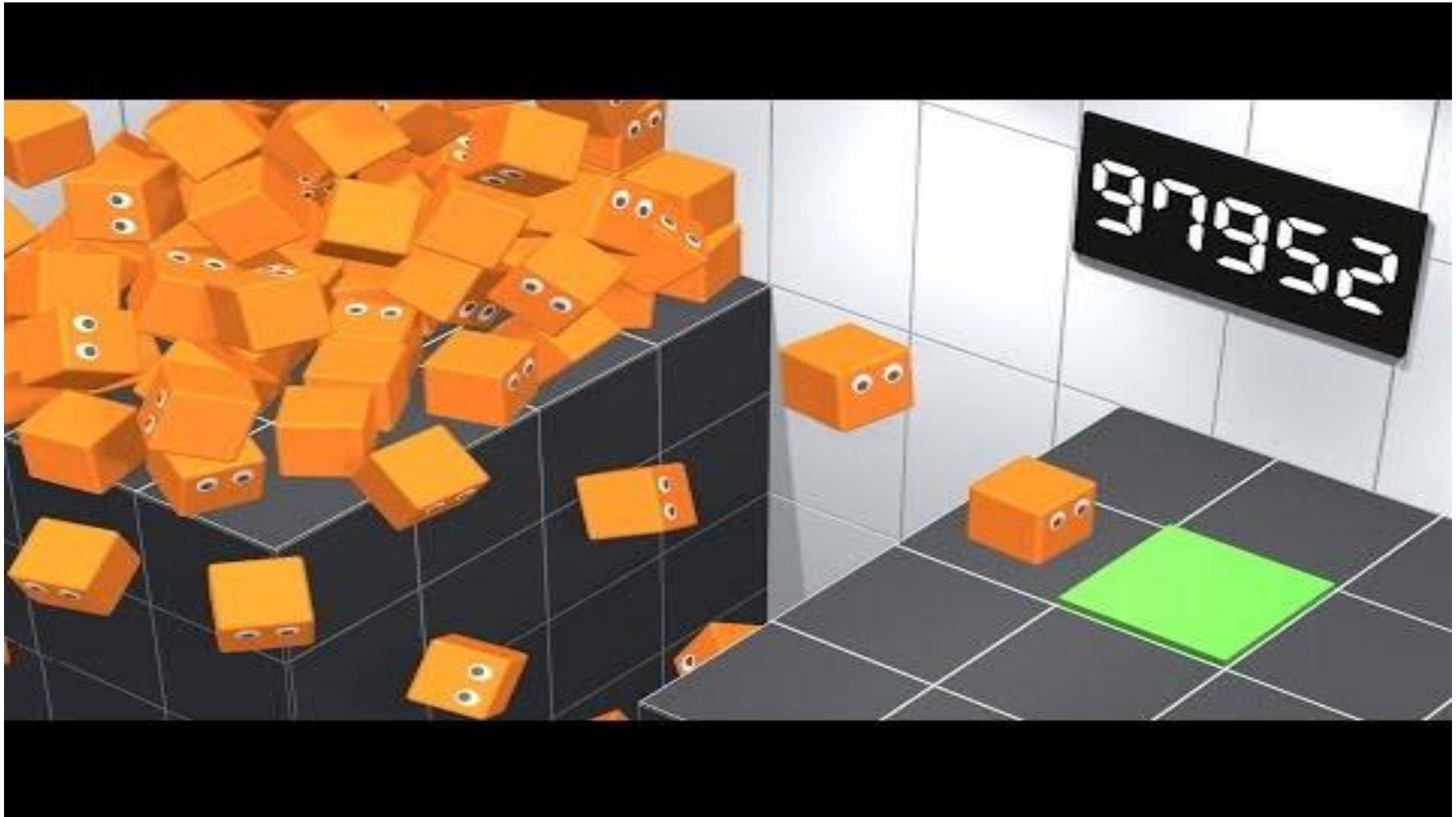


<https://www.youtube.com/watch?v=TmPfTpjtdgg>

방탈출 게임 강화학습 적용



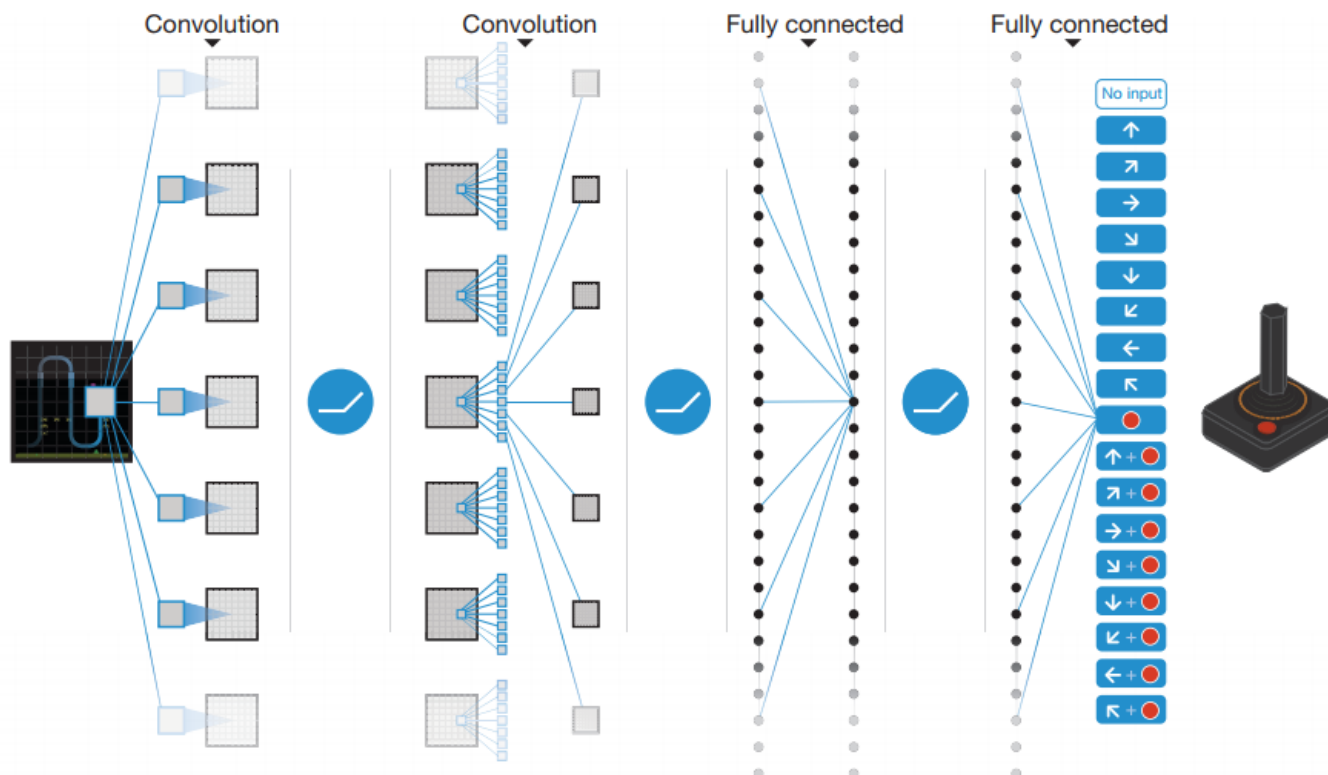
지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.



<https://www.youtube.com/watch?v=v3UBIEJDXR0>

■ 딥러닝 + 강화학습 알고리즘 결합

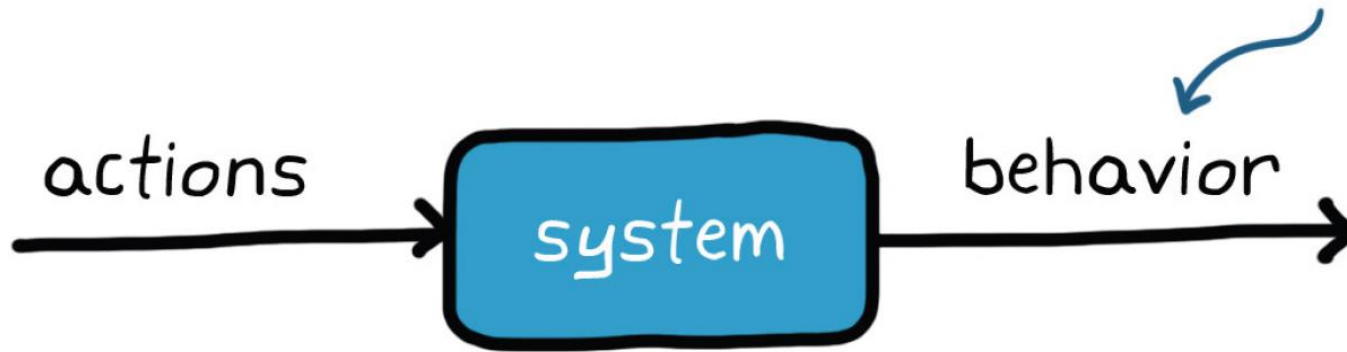
- 컨볼루션 신경망(CNN, 딥러닝의 일종)과 Q-학습(강화학습 알고리즘)을 결합한 모델
- 입력층은 게임 스크린 영상(최근 4장의 영상 $84 \times 84 \times 4$ 텐서) ← 이 텐서가 상태를 표현
- 출력층은 18개 노드(게임마다 4~18개 조이스틱 행동이 있는데 게임마다 필요한 만큼 사용)



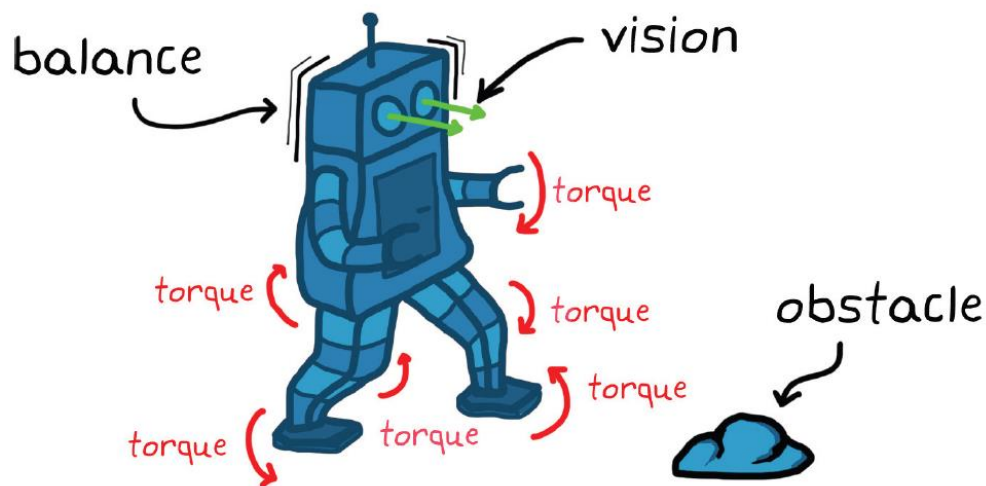
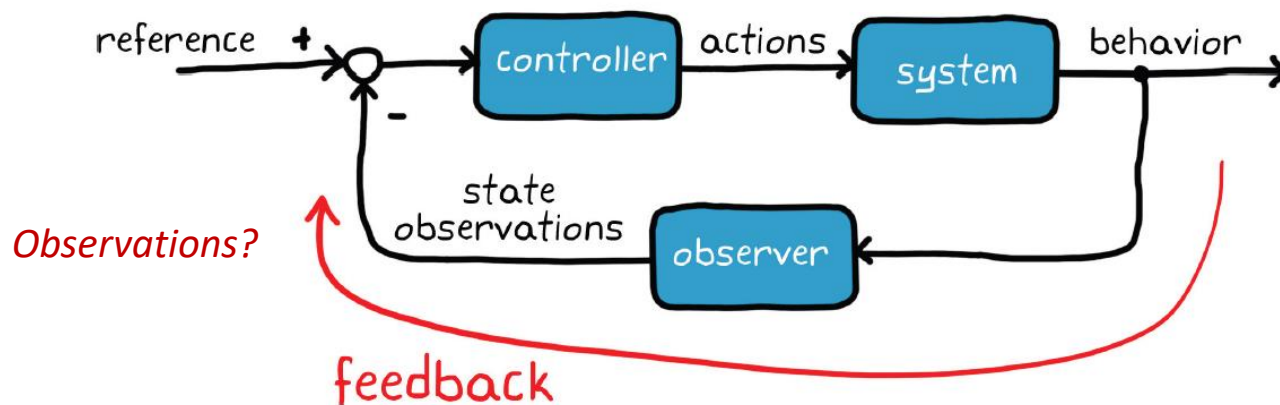
Goal to Control



which actions generate the desired behavior?



이쪽 보행 로봇에서의 Actions?



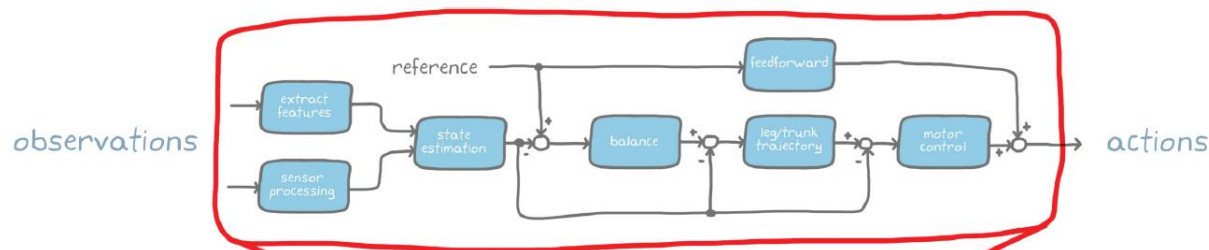
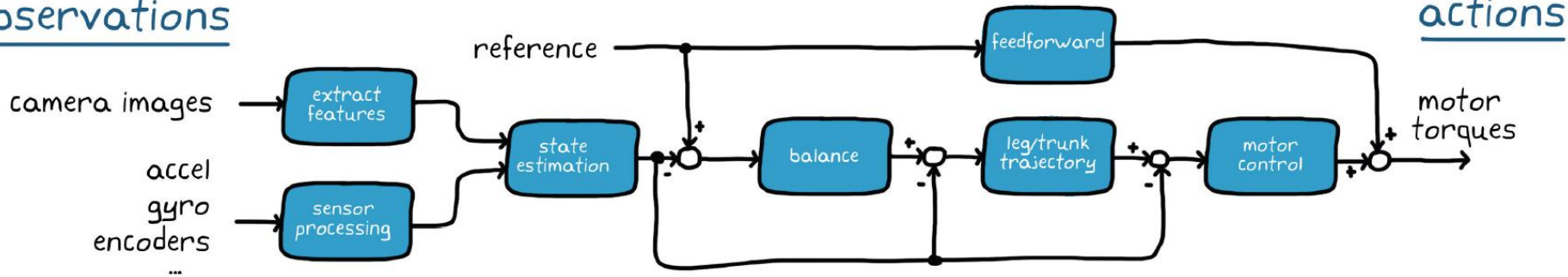
어떻게 Controller가 Action 값을 조절할 것인가?

제어공학 vs 강화학습



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

observations



what does this function look like?

how do you design it?

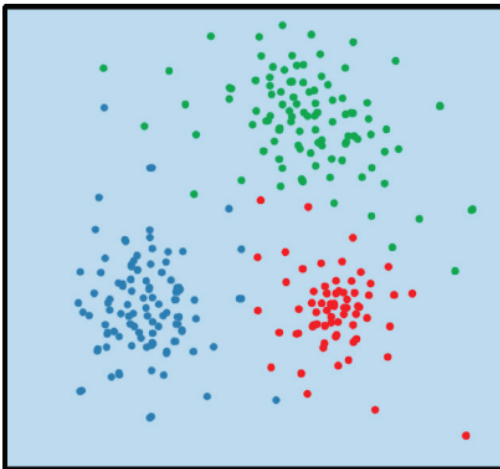
Reinforcement Learning!

machine learning

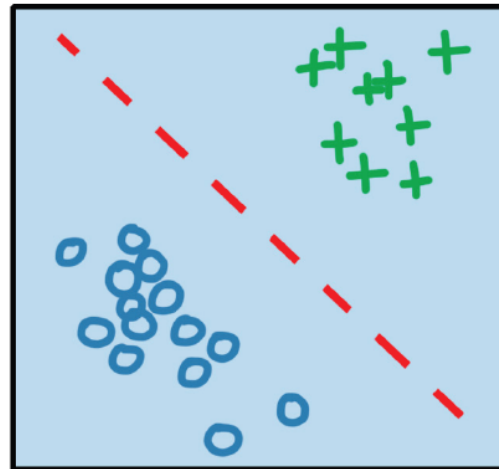
unsupervised
learning

supervised
learning

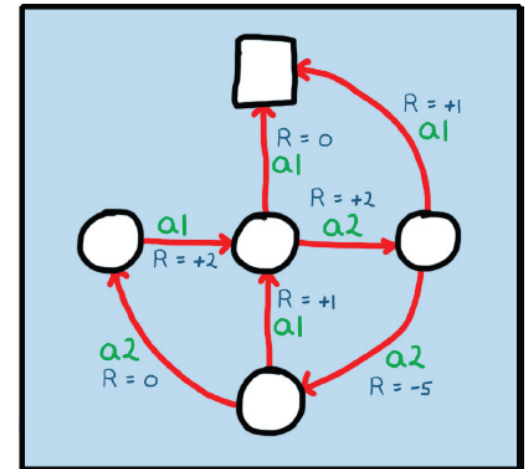
reinforcement
learning



목표: Labeling



목표: Classification



목표??

강화학습 목표와 Training 프로세스

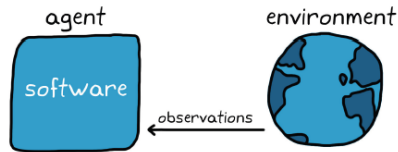


지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

- 목표: Find the best actions that will generate the optimal outcome

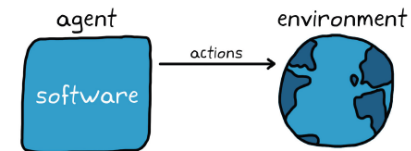
1

The agent is able to observe the current state of the environment.



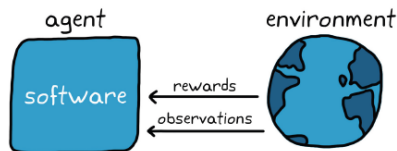
2

From the observed state, it decides which action to take.



3

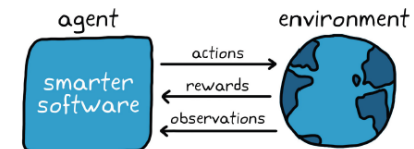
The environment changes state and produces a reward for that action. Both of which are received by the agent.



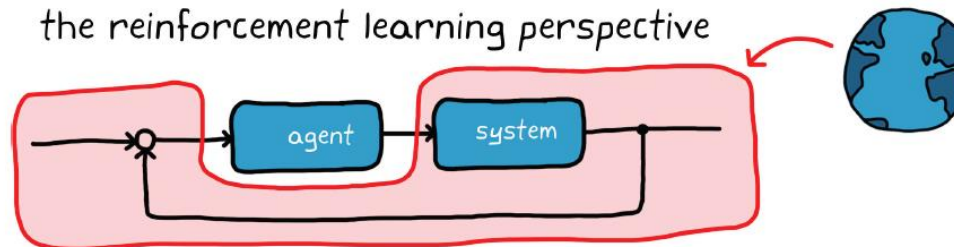
4

Using this new information, the agent can determine whether that action was good and should be repeated, or if it was bad and should be avoided.

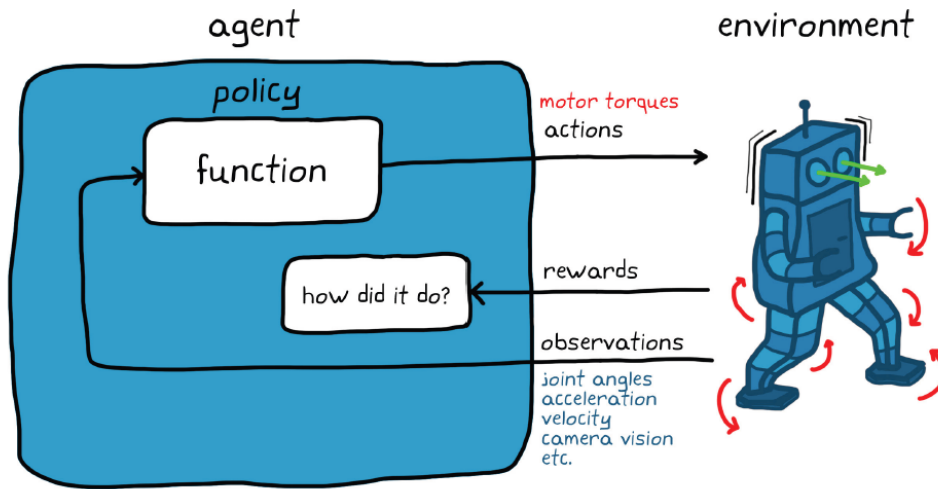
The observation-action-reward cycle continues until learning is complete.



the reinforcement learning perspective



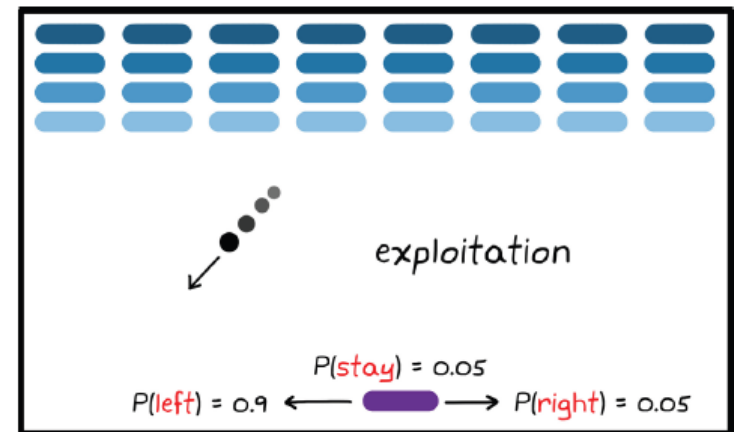
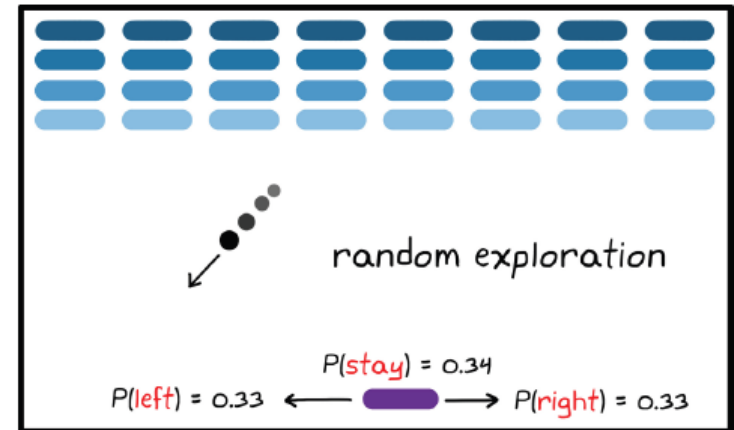
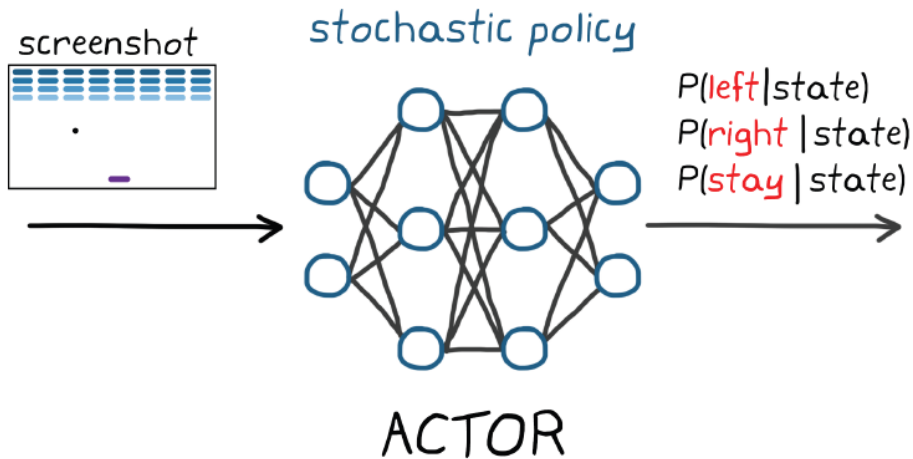
Policy란?



$$\text{policy} \\ \text{actions} = \text{function}(\text{state observations})$$

Function을 무엇으로 구현할까?

Policy를 신경망으로 구현



강화학습 구현을 위한 순서



1. Environment 구축

- Action에 따라 서로 다른 Observation을 얻을 수 있는 Environment 구축 (테스트베드 or Simulation)

2. Reward 설계

- 사람이 직접 알고리즘을 설계 (이론적/경험적 지식에 따라)

3. Policy 설계

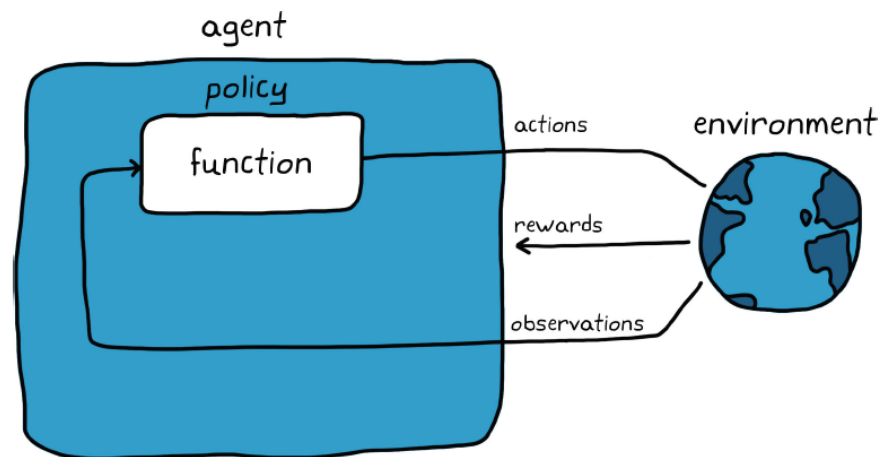
- 어떤 Policy 모델과 옵션을 사용할 것인지

4. Training

- Agent 훈련

5. Deploy

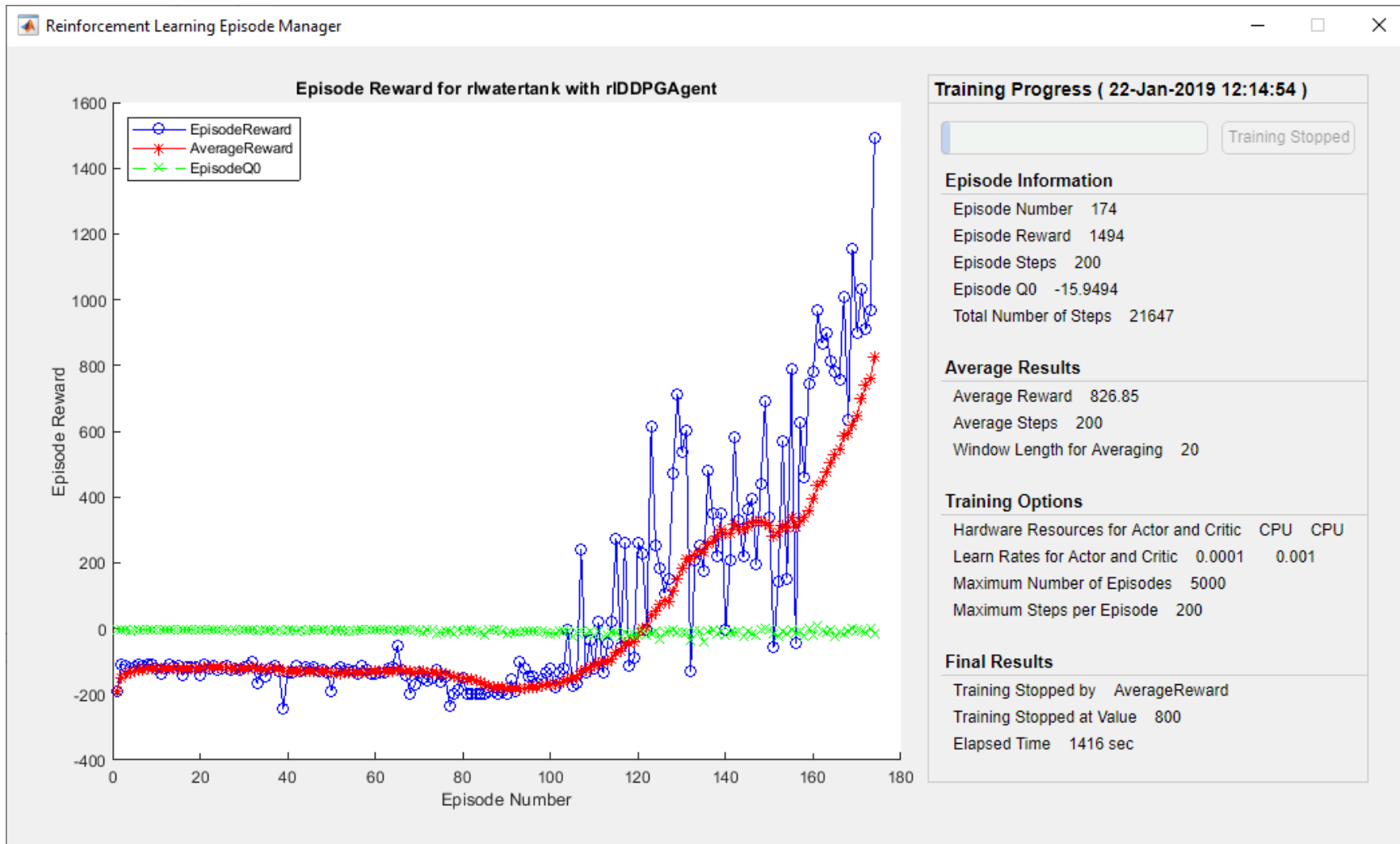
- 훈련이 완료된 Agent를 실제 시스템에 적용

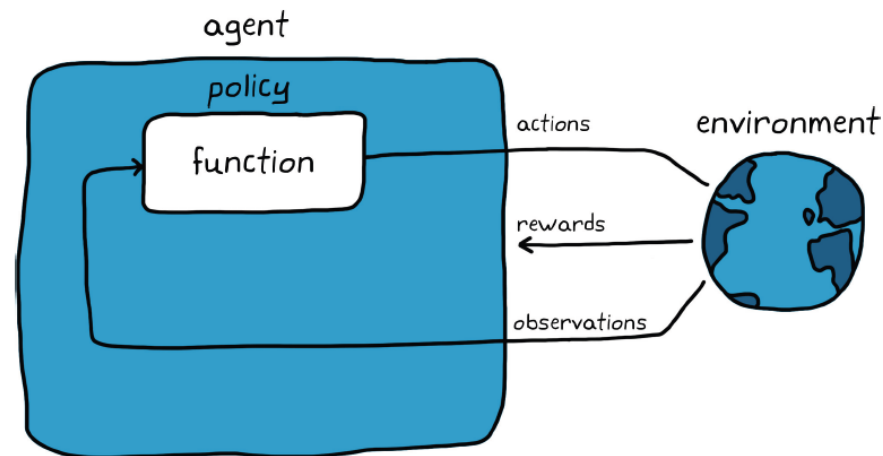
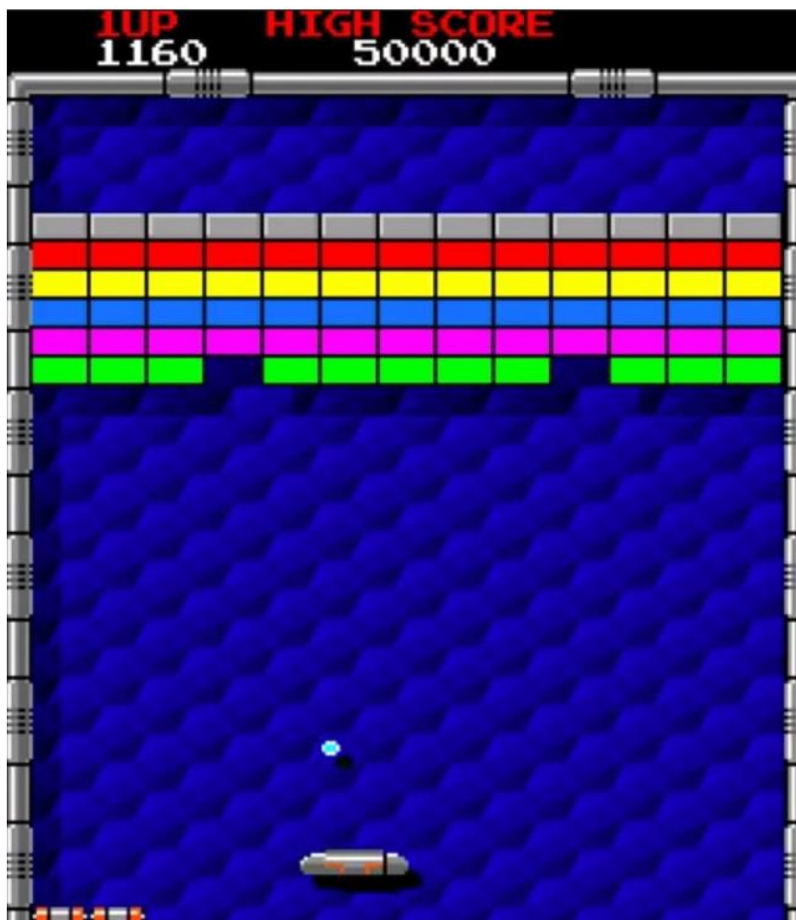


강화학습 Training 결과 예시



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.





1. Action을 어떻게 설정하면 좋을까?
2. Observation을 어떻게 설정하면 좋을까?
3. Environment는 어떻게 구축 해야할까?
4. Reward를 어떻게 설계하면 좋을까?

1. 머신러닝 + 딥러닝 개념을 이해한 상태여야만 입문할 수 있다.
2. 도메인 적용을 위해서는 도메인에 대한 전문적인 지식이 요구된다.



강화학습 && 도메인 지식 => 강화학습 응용 가능

Ex) 알파고 제로 (강화학습 기반) 개발 당시,
프로 기사 출신의 박사가 개발팀에 있었음.

1. 도메인 전문가가 강화학습을 공부하거나,
2. 강화학습 전문가가 도메인을 공부하거나

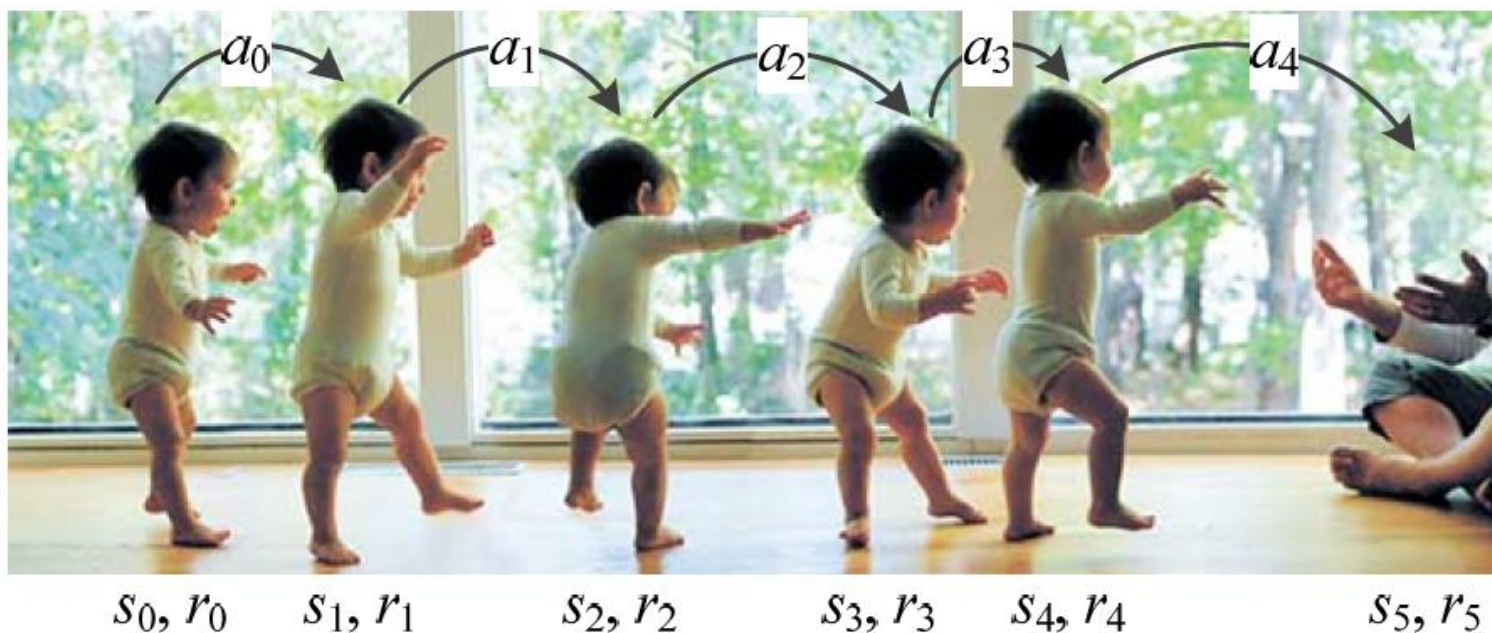
■ 강화 학습의 핵심 연산

- 시간에 따른 상태state, 행동action, 보상reward의 순차적 처리 $t=0, 1, 2, 3, \dots, T$

- $s_0, r_0 \xrightarrow{a_0} s_1, r_1 \xrightarrow{a_1} s_2, r_2 \xrightarrow{a_2} \dots \xrightarrow{a_{T-1}} s_T, r_T$

- 식으로 쓰면,

$$\text{강화 학습의 핵심 연산 } f: (s_t, a_t) \rightarrow (s_{t+1}, r_{t+1}) \quad (9.1)$$



[그림 9-2] 강화 학습의 핵심 개념인 상태 s_i , 행동 a_i , 보상 r_i

■ 에이전트와 agent 환경 environment

- 에이전트는 정책을 가지고 행동을 결정(정책은 학습으로 알아내야 함)
- 환경은 MDP를 가지고 상태 전환과 보상액 결정(MDP는 주어지는 것: 지도 학습의 훈련집합에 비유할 수 있음)

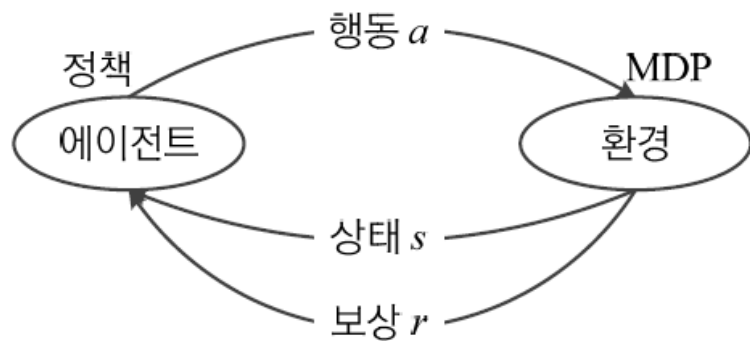


그림 9-3 강화 학습의 계산 모델

■ 강화 학습의 목표는 누적 보상액을 최대화하는 것

- 이를 달성하려면 매 순간 좋은 행동을 취할 수 있어야 함 → 정책에 따라 행동을 결정하므로 좋은 정책이 필요한 셈
- 강화 학습은 주어진 MDP를 가지고 최적 정책을 찾아야 함(지도 학습이 훈련집합을 가지고 최적 매개변수를 찾는 것에 비유할 수 있음)

■ 상태, 행동, 보상의 표현

- 대부분 응용에서는 이산 값을 가짐
- 상태값 집합 $\mathcal{S} = \{s_1, s_2, \dots, s_n\}$, 행동값 집합 $\mathcal{A} = \{a_1, a_2, \dots, a_m\}$, 보상값 집합 $\mathcal{R} = \{r_1, r_2, \dots, r_l\}$ 으로 표기
- $P(s_{t+1} = s_{58} | s_t = s_{122})$: t 순간에 s_{122} 상태에 있다가 다음 순간에 s_{58} 상태로 바뀔 확률
- $P(s_{t+1} = s_{58} | s_t = s_{122}, a_t = a_2)$: t 순간에 s_{122} 상태에서 행동 a_2 를 취했을 때 s_{58} 상태로 바뀔 확률

예시 환경: Grid World

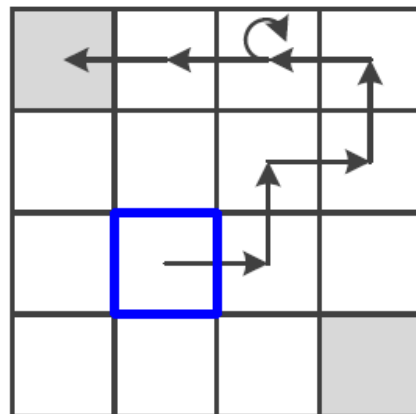


예제 9-1

격자 세계

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

(a) 예제 격자 보드



(b) 이동 경로의 예

<Action>

- 동서남북: 4가지

<보상>

- 회색 칸에 들어가면: +5
- 흰색 칸에 들어가면: 0
- 밖으로 나가면: -1

그림 9-4 격자 세계

$$\mathcal{S} = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16\}$$

$$\mathcal{A} = \{\text{동, 서, 남, 북}\}$$

$$\mathcal{R} = \{0, 5, -1\}$$

$$s_0, r_0 \xrightarrow{a_0} s_1, r_1 \xrightarrow{a_1} s_2, r_2 \xrightarrow{a_2} \dots \xrightarrow{a_{T-1}} s_T, r_T$$

(b) 이동 경로에 대해서 표현해보면=>

총 누적 보상은?

Policy (정책)의 개념



■ 정책 policy

- 에이전트는 확률 규칙으로 행동을 결정하는데, 이 규칙을 정책이라 부름
- [예제 9-1]이 동, 서, 남, 북이라는 4가지 행동에 대해 동일 확률을 사용한다고 가정하면, 이 정책을 다음과 같이 쓸 수 있음

[예제 9-1]의 정책:

$$P(a = \text{동} | s = i) = P(a = \text{서} | s = i) = P(a = \text{남} | s = i) = P(a = \text{북} | s = i) = \frac{1}{4}, \quad i = 2, 3, \dots, 15$$

해당 정책이 좋은 정책인가?

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

<보상>

- 회색 칸에 들어가면: +5
- 흰색 칸에 들어가면: 0
- 밖으로 나가면: -1

(a) 예제 격자 보드

Exploration vs Exploitation



잭팟이 터질 확률은 Bandit-Machine 마다 서로 다름



그림 9-6 k-손잡이 밴딧 문제

총 20 번의 기회가 있다고 가정 => 어떻게 시도해야 최대한의 보상을 받을 수 있을까?

랜덤으로 골라서 시도하다가 12번 만에 2번 손잡이에서 잭팟이 터졌다고 가정하면,

3, 4, 1, 2, 5, 4, 3, 1, 2, 5, 1, 2

13 번째 시도부터는 무엇을 고르는 것이 현명할까?

Exploration vs Exploitation



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

확률 공개

1/100

1/500

1/300

1/10

1/2000



그림 9-6 k-손잡이 밴딧 문제

극단적인 Exploitation을 취하면,

3, 4, 1, 2, 5, 4, 3, 1, 2, 5, 1, **2**, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, ... 탐사 방식

극단적인 Exploration을 취하면, (**무조건 랜덤**)

3, 4, 1, 2, 5, 4, 3, 1, 2, 5, 1, **2**, 3, 5, 4, 3, 1, 2, 5, 3, 1, 2, ... 탐험 방식

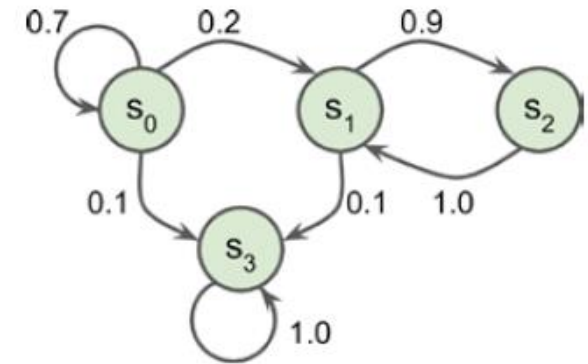
적당히 Exploitation 하면서, 간간히 Exploration 하는게 좋지 않을까?

■ Step (단계)

- State 별 Discrete한 시간을 의미
 - t : 현재, $t-1$: 현재 바로 직전 단계

■ State (상태)

- 이산시간 별 환경의 상태를 나타내는 벡터
 - $S(t)$: 현재 상태벡터, $S(t-1)$: 바로 직전 상태벡터



Markov Chain 예시

■ Markov Property

- $S(t)$ 는 오직 $S(t-1)$ 와 상태 전이 확률에 따라서만 결정

■ Markov Chain

- Markov Property를 지닌 이산시간의 확률 과정

■ 마르코프 성질

- 행동을 결정할 때 이전 이력이 중요하지 않다면 마르코프 성질을 만족함
 - [예제 9-1]의 격자 세계와 [그림 9-7]의 바둑은 마르코프 성질을 만족

*Markov Property를 만족하지 않는 문제는
어떤 것이 있을까?*

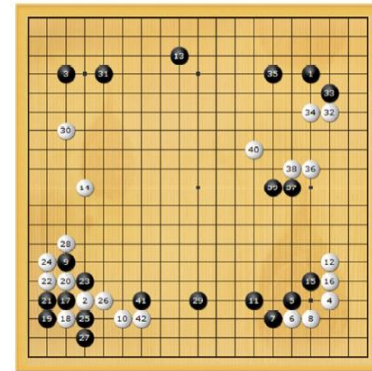
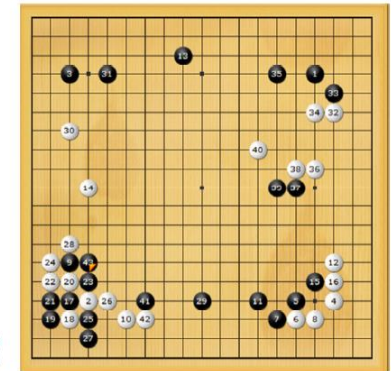


그림 9-7 바둑

(14행, 4열)



■ 마르코프 성질을 수식으로 표현하면,

$$P(s_{t+1}, r_{t+1} | s_0, a_0, r_1, s_1, a_1, \dots, r_{t-1}, s_{t-1}, a_{t-1}, r_t, s_t, a_t) = P(s_{t+1}, r_{t+1} | s_t, a_t) \quad (9.2)$$

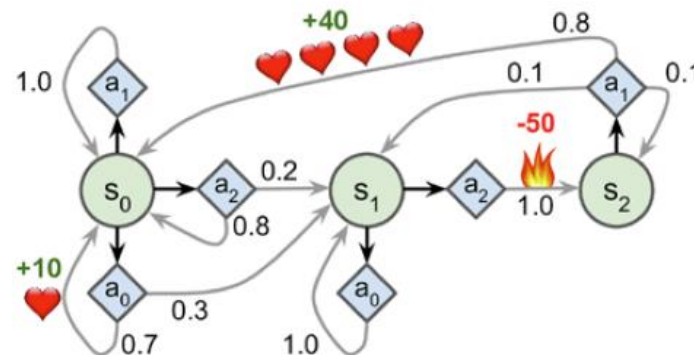
- 첨자를 생략하여 간략히 표기하면,

$$P(s_{t+1}, r_{t+1} | s_t, a_t) = P(s', r | s, a) \Leftarrow \text{MDP의 Dynamics} \quad (9.3)$$

Markov Property를 크게 벗어나는 상황에서는 강화 학습을 적용하기 힘들

■ 마르코프 결정 과정 (MDP)

- Richard Bellman이 처음으로 기술
- 마르코프 체인 개념에 action과 reward 개념을 도입



MDP의 Dynamics

MDP 확률분포: $P(s', r | s, a)$, $\forall s \in \mathcal{S}, \forall a \in \mathcal{A}, \forall s' \in \mathcal{S}, \forall r \in \mathcal{R}$

강화학습은 주어진 MDP 하에서

“최대의 보상”을 얻을 수 있는 “최적의 정책”을 찾는 것



지도 학습에서 Cost function 이 최소가 되는 해(Weight)를 찾는 것과 유사



“지도 학습에서의 Cost function” 처럼

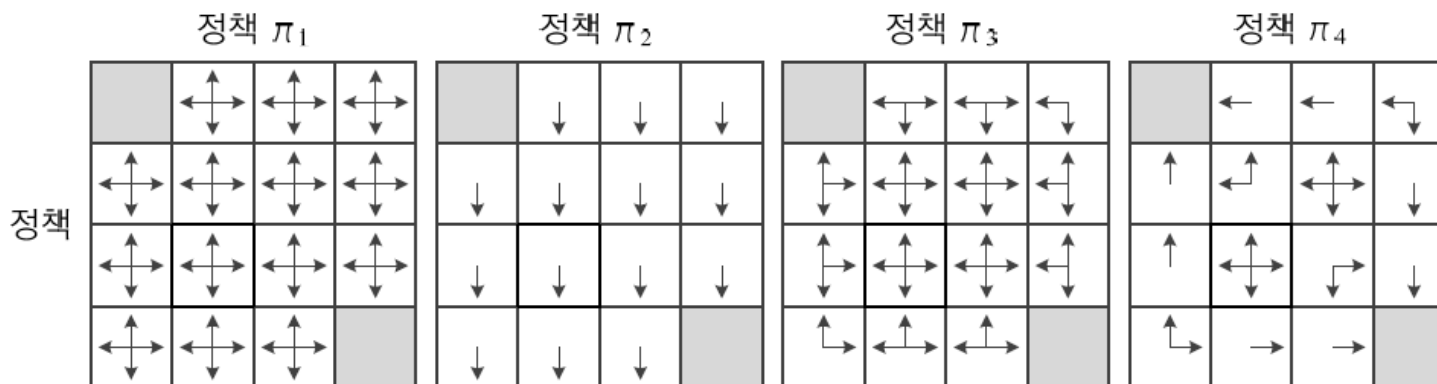
최적의 정책이라는 것을 표현할 수 있는 함수가 필요

- 정책 π 는 상태 s 에서 행동 a 를 취할 확률을 명시한 것

$$\pi(a|s) = P(a|s), \forall s, \forall a$$

<보상>

- 회색 칸에 들어가면: +5
- 흰색 칸에 들어가면: -1
- 밖으로 나가면: -2



어느 정책이 최적인가?

- 상태 가치함수는 특정 정책의 좋은 정도를 측정하는 함수
 - 좋은 정도는 상태 s 로부터 종료 상태에 이르기까지 누적 보상액의 추정치

- 최적의 정책을 가치 함수로 표현하면,

$$\hat{\pi} = \operatorname{argmax}_{\pi} v_{\pi}(s), \forall s \in \mathcal{S} \quad (9.8)$$

- 가치함수 $v_{\pi}(s)$ 를 구하는 식

- $P(z)$ 는 경로 z 의 발생 확률, G_t 는 경로 z 의 "예상" 누적 보상 일 때,

$$v_{\pi}(s) = E[G_t] = \int G_t P(a_t, s_{t+1}, a_{t+1}, s_{t+2}, \dots | s_t) da_t: a_T$$

가치함수는 누적보상에 대한 기대값

$$v_{\pi}(s) = \sum_{s \text{에서 출발하는 모든 경로 } z} G_t P(z) \quad \text{이산확률분포함수 일 때,}$$

- 예상 누적 보상액을 계산하는 식

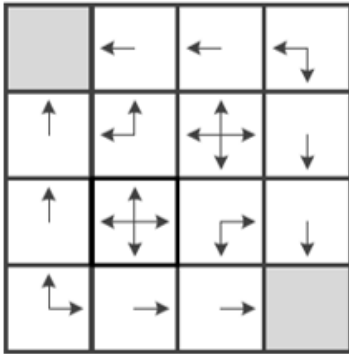
$$G_t = r_{t+1} + r_{t+2} + \dots + r_T \quad (9.10)$$

State Value Function 예제



■ 예제)

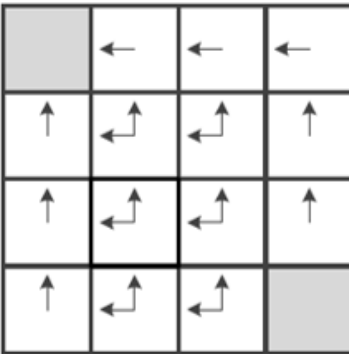
정책 π_4



$$v_{\pi_4}(s = 2) =$$

$$v_{\pi_4}(s = 11) =$$

정책 π_5



$$v_{\pi_5}(s = 2) =$$

$$v_{\pi_5}(s = 11) =$$

Value Function 순환식



정책 π_5

	←	←	←
↑	↖	↖	↑
↑	↖	↖	↑
↑	↖	↖	

$$\begin{aligned}
 v_{\pi_5}(s = 11) &= \frac{1}{2} \frac{1}{2} (-1 - 1 - 1 + 5) + \frac{1}{2} \frac{1}{2} \frac{1}{2} (-1 - 1 - 1 + 5) + \frac{1}{2} \frac{1}{2} \frac{1}{2} (-1 - 1 - 1 + 5) \\
 &\quad + \frac{1}{2} \frac{1}{2} \frac{1}{2} (-1 - 1 - 1 + 5) + \frac{1}{2} \frac{1}{2} \frac{1}{2} (-1 - 1 - 1 + 5) + \frac{1}{2} \frac{1}{2} (-1 - 1 - 1 + 5) \\
 &\quad \Downarrow \\
 &= \frac{1}{2} (-1 + v_{\pi_5}(7)) + \frac{1}{2} (-1 + v_{\pi_5}(10))
 \end{aligned}$$

- a : s 에서 현재 정책 π 에 의해 결정되는 행동
- s' : s 에서 a 를 수행했을 때 얻는 상태
- r : s 에서 a 를 수행했을 때 얻는 보상



■ 현재 상태 s 에서의 현재 정책 π 에 대한 가치를 표현하는 함수

$$v_{\pi}(s) = \sum_{s \text{에서 출발하는 모든 경로 } z} P(z) \mathbb{E}(z) = \sum_{a \in \mathcal{A}(s)} P(a|s) (r + v_{\pi}(s')), \quad \forall s \in \mathcal{S}$$

Discount Rate (할인율)



- 예상 누적 보상액을 계산하는 식 (에피소드의 경로가 무한할 때)

$$G_t = r_{t+1} + r_{t+2} + \dots$$

누적 보상액을 위와 같이 표현할 경우 발생하는 문제점?

- 할인율을 적용한 예상 누적 보상액

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots = \sum_{k=1}^{\infty} \gamma^{k-1} r_{t+k} \quad (9.11)$$

$$G_t = r_{t+1} + \gamma G_{t+1}$$

- 할인율을 적용한 현 상태에 대한 가치 함수

$$v_{\pi}(s) = \sum_{a \in \mathcal{A}(s)} P(a|s)(r + v_{\pi}(s')), \quad \forall s \in \mathcal{S}$$



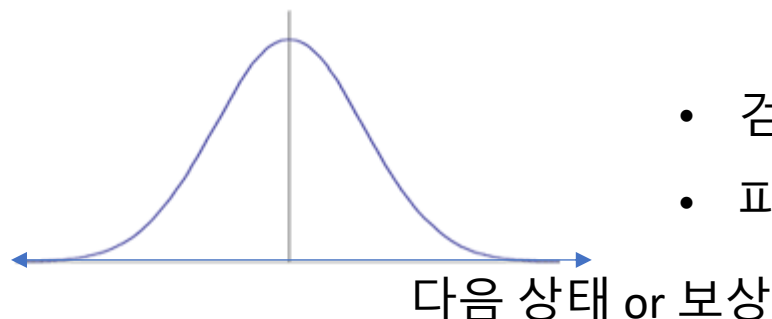
$$v_{\pi}(s) = \sum_{a \in \mathcal{A}(s)} P(a|s)(r + \gamma v_{\pi}(s')), \quad \forall s \in \mathcal{S}$$

Deterministic vs Stochastic



MDP의 Dynamics

MDP 확률분포: $P(s', r | s, a), \forall s \in \mathcal{S}, \forall a \in \mathcal{A}, \forall s' \in \mathcal{S}, \forall r \in \mathcal{R}$



- 검정색 선: Dynamics가 1
- 파란색 선: 확률 분포를 가질 때

**환경 모델의 세부적인
정보를 알아야만 함**

■ Deterministic MDP

- MDP의 Dynamics가 1인 경우
- 현재 상태에서 정책에 의해 결정된 행동을 하면, 다음 상태와 보상이 1가지로 확정됨.

■ Stochastic MDP

- MDP의 Dynamics가 확률 분포로 존재하는 경우
- 현재 상태에서 정책에 의해 결정된 행동을 취하면, 다음 상태와 보상이 확률적으로 결정됨
=> 즉, 현재 상태에서 정책에 의해 결정되는 행동이 확률적이라 생각할 수 있음.

Bellman Equation



■ Deterministic MDP에서의 “상태” 가치 함수

이산 확률 분포

$$v_{\pi}(s) = \sum_{a \in \mathcal{A}(s)} P(a|s)(r + \gamma v_{\pi}(s')), \forall s \in \mathcal{S}$$

Physical meaning:

■ Stochastic MDP에서의 “상태” 가치 함수

$$v_{\pi}(s) = \sum_{a \in \mathcal{A}(s)} P(a|s) \sum_{s'} \sum_r P(s', r|s, a)(r + \gamma v_{\pi}(s')), \forall s \in \mathcal{S}$$

Bellman Equation

Physical meaning:

■ Stochastic MDP에서의 “상태” 가치 함수

$$v_{\pi}(s) = \sum_{a \in \mathcal{A}(s)} P(a|s) \sum_{s'} \sum_r P(s', r|s, a) (r + \gamma v_{\pi}(s')), \forall s \in \mathcal{S}$$

행동 가치 함수: $q_{\pi}(s, a)$

$$v_{\pi}(s) = \sum_{a \in \mathcal{A}(s)} p(a|s) q_{\pi}(s, a)$$

■ Stochastic MDP에서의 “행동” 가치 함수

$$q_{\pi}(s, a) = \sum_{s'} \sum_r P(s', r|s, a) (r + \gamma v_{\pi}(s')), \forall s \in \mathcal{S}, \forall a \in \mathcal{A}$$

Optimal Policy Function



- 최적 정책을 찾는 식을 상태 가치함수를 이용하여 표현하면,

$$\hat{v}(s) = v_{\hat{\pi}}(s) = \max_{\pi} v_{\pi}(s), \forall s \in \mathcal{S}$$

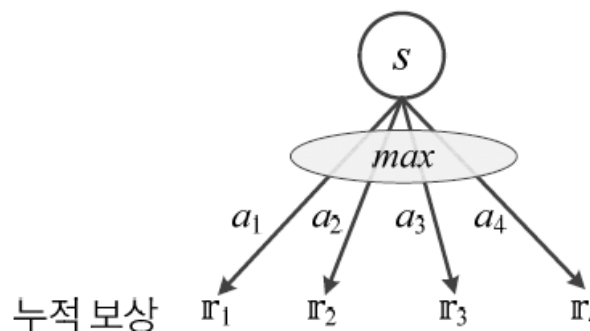
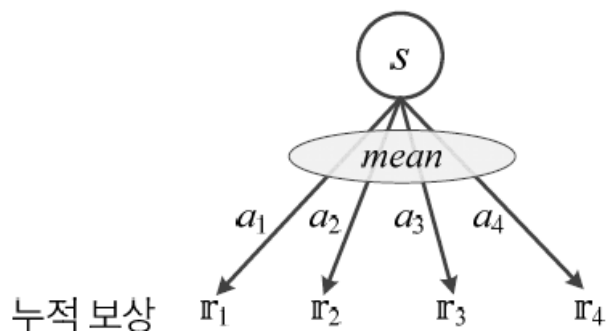
$$v_{\pi}(s) = \sum_{a \in \mathcal{A}(s)} P(a|s) \sum_{s'} \sum_r P(s', r|s, a) (r + \gamma v_{\pi}(s')), \forall s \in \mathcal{S}$$

- 미래 가치를 최대로 하는 액션을 현재 상태에서 선택하는 것

$$\hat{v}(s) = \max_{a \in \mathcal{A}(s)} \sum_{s'} \sum_r P(s', r|s, a) (r + \gamma \hat{v}(s')), \quad \forall s \in \mathcal{S} \quad (9.18)$$

행동 가치 함수: $q_{\pi}(s, a)$

즉, 현재 상태에서의 행동 가치가 가장 커지는 액션을 선택하는 것이 최적의 정책



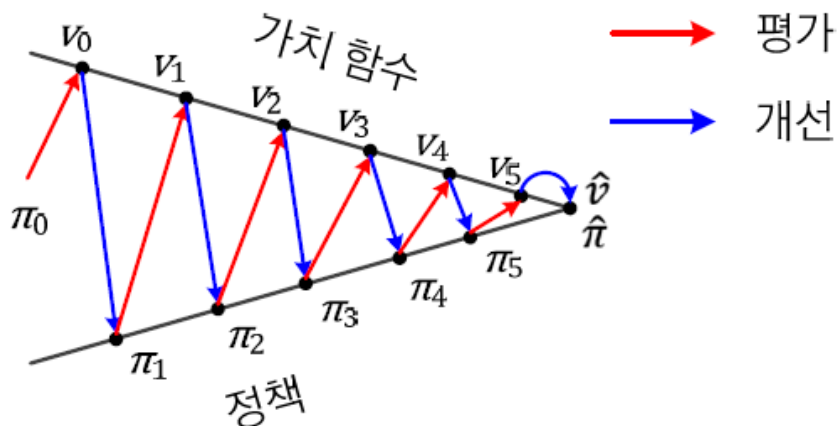
강화 학습이 Optimal Policy를 찾는 과정



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

■ $\hat{v}(s) = \max_{a \in \mathcal{A}(s)} \sum_{s'} \sum_r P(s', r | s, a) (r + \gamma \hat{v}(s')), \quad \forall s \in \mathcal{S}$ 어떻게 구하나?

- 상태들이 순환적으로 연결되어 있는데, 어느 상태도 값이 정해져 있지 않음
- 이들을 푸는 보편적 방법
 - 정책을 임의로 설정하고 출발 $\rightarrow$ 가치함수 계산 $\rightarrow$ 정책 개선 $\rightarrow$ 가치함수 계산 $\rightarrow$...
 - 가치 반복 알고리즘, 몬테카를로 방법, 시간차 학습 등이 있음.



**최적의 가치함수를 찾으면서,
정책도 함께 갱신 하다보면
결국 최적의 정책을 찾게 된다.**

그림 9-11 강화 학습이 최적 정책을 찾아가는 보편적 과정

■ 가치 반복 알고리즘의 전략

동적 프로그래밍

- MDP를 보고 최적 가치함수를 찾은 후, 최적 가치함수로부터 최적 정책을 찾음

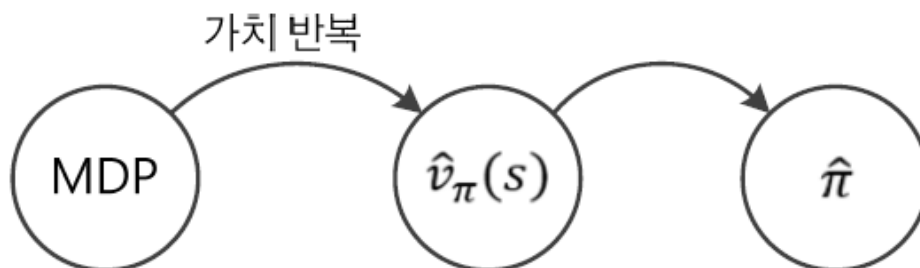


그림 9-13 가치 반복 알고리즘

■ MDP에 따라 가치함수를 더 좋은 가치함수로 갱신하는 식

$$v^{(t+1)}(s) = \max_{a \in \mathcal{A}(s)} \sum_{s'} \sum_r \boxed{P(s', r | s, a)} (r + v^{(t)}(s')), \forall s \in \mathcal{S}$$

MDP의 Dynamics를 알고 있어야만 적용가능

Value iteration 알고리즘



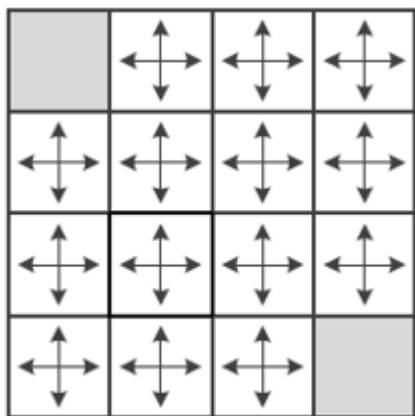
알고리즘 9-2 가치 반복

입력: MDP, 즉 $P(s', r|s, a), \forall s \in \mathcal{S}, \forall s' \in \mathcal{S}, \forall a \in \mathcal{A}, \forall r \in \mathcal{R}$, 임계값 ε

출력: 최적 정책 $\hat{\pi}$ 과 최적 가치함수 $\hat{v}$

```
1  for ( $s \in \mathcal{S}$ )  $v^{(0)}(s) = 0$  // 초기화 (0 또는 임의값으로 설정함)
2   $t = 0$ 
3  repeat
4       $max\_current = 0$ 
5      for ( $s \in \mathcal{S}$ )
6           $v^{(t+1)}(s) = \max_{a \in \mathcal{A}(s)} \sum_{s'} \sum_r P(s', r|s, a)(r + v^{(t)}(s'))$  // 식 (9.19)
7          라인 6에서 max였던 행동집합을  $A(s)$ 에 대입한다.
8           $max\_current = \max(max\_current, v^{(t+1)}(s) - v^{(t)}(s))$ 
9       $t++$ 
10 until ( $max\_current < \varepsilon$ )
11  $\hat{v} = v^{(t)}$ 
12 for ( $s \in \mathcal{S}$ ) //  $A(s)$  정보를 가지고 최적 정책 구축
13      $\sum_{a \in A(s)} P(a|s) = 1$ 이 되도록  $A(s)$ 에 속한 행동  $a$ 에 확률을 배분한 후 확률을  $\hat{\pi}$ 에 저장한다.
```


Value iteration 알고리즘 예제



<보상>

- 회색 칸에 들어가면: +5
- 흰색 칸에 들어가면: -1
- 밖으로 나가면: -2

$$\begin{aligned}
 v^{(1)}(2) &= \max_{a \in \{\text{동, 서, 남, 북}\}} \sum_{s'} \sum_r P(s', r | 2, a) (r + v^{(0)}(s')) \\
 &= \max\{P(3, -1 | 2, \text{동})(-1 + v^{(0)}(3)), P(1, 5 | 2, \text{서})(5 + v^{(0)}(1)), \\
 &\quad P(6, -1 | 2, \text{남})(-1 + v^{(0)}(6)), P(2, -2 | 2, \text{북})(-2 + v^{(0)}(2))\} \\
 &= \max\{1 * (-1 + 0), \underline{1 * (5 + 0)}, 1 * (-1 + 0), 1 * (-2 + 0)\} = 5
 \end{aligned}$$

$q(2, \text{서})$

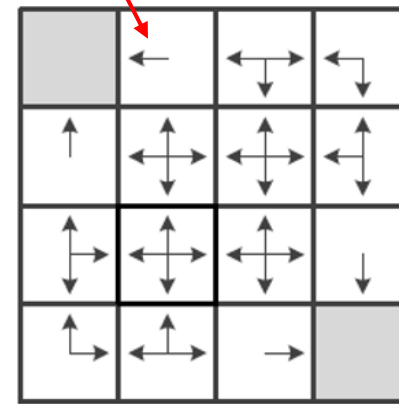
$$\begin{aligned}
 v^{(1)}(6) &= \max_{a \in \{\text{동, 서, 남, 북}\}} \sum_{s'} \sum_r p(s', r | 6, a) (r + v^{(0)}(s')) \\
 &= \max\{P(7, -1 | 6, \text{동})(-1 + v^{(0)}(7)), P(5, -1 | 6, \text{서})(-1 + v^{(0)}(5)), \\
 &\quad P(10, -1 | 6, \text{남})(-1 + v^{(0)}(10)), P(2, -1 | 6, \text{북})(-1 + v^{(0)}(2))\} \\
 &= \max\{1 * (-1 + 0), 1 * (-1 + 0), 1 * (-1 + 0), 1 * (-1 + 0)\} = -1
 \end{aligned}$$

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

초기 최적 가치

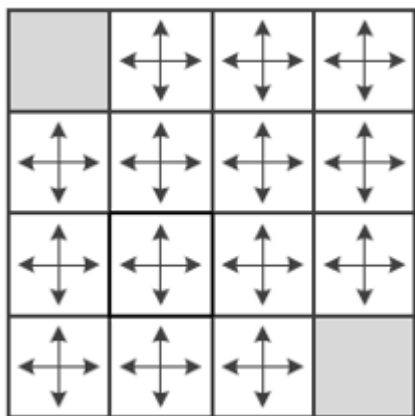
0	5	-1	-1
5	-1	-1	-1
-1	-1	-1	5
-1	-1	5	0

i=1 일 때, 최적 가치



i=1 일 때, 정책

Value iteration 알고리즘 예제



$$\begin{aligned}
 v^{(2)}(2) &= \max_{a \in \{\text{동, 서, 남, 북}\}} \sum_{s'} \sum_r P(s', r | 2, a) (r + v^{(1)}(s')) \\
 &= \max\{ P(3, -1 | 2, \text{동}) (-1 + v^{(1)}(3)), P(1, 5 | 2, \text{서}) (5 + v^{(1)}(1)), \\
 &\quad P(6, -1 | 2, \text{남}) (-1 + v^{(1)}(6)), P(2, -2 | 2, \text{북}) (-2 + v^{(1)}(2)) \} \\
 &= \max\{ 1 * (-1 - 1), 1 * (5 + 0), 1 * (-1 - 1), 1 * (-2 + 5) \} = 5
 \end{aligned}$$

$$\begin{aligned}
 v^{(2)}(6) &= \max_{a \in \{\text{동, 서, 남, 북}\}} \sum_{s'} \sum_r P(s', r | 6, a) (r + v^{(1)}(s')) \\
 &= \max\{ P(7, -1 | 6, \text{동}) (-1 + v^{(1)}(7)), P(5, -1 | 6, \text{서}) (-1 + v^{(1)}(5)), \\
 &\quad P(10, -1 | 6, \text{남}) (-1 + v^{(1)}(10)), P(2, -1 | 6, \text{북}) (-1 + v^{(1)}(2)) \} \\
 &= \max\{ 1 * (-1 - 1), 1 * (-1 + 5), 1 * (-1 - 1), 1 * (-1 + 5) \} = 4
 \end{aligned}$$

<보상>

- 회색 칸에 들어가면: +5
- 흰색 칸에 들어가면: -1
- 밖으로 나가면: -2

Bootstrap 방식:

다른 상태에서의 가치를

보고 정책을 갱신

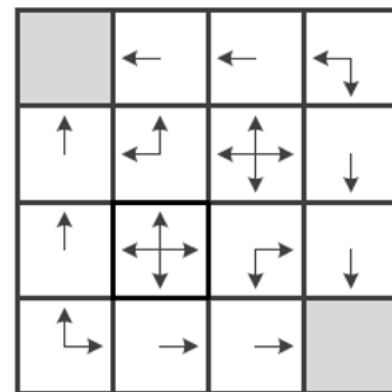
0	5	-1	-1
5	-1	-1	-1
-1	-1	-1	5
-1	-1	5	0

i=1 일 때, 최적 가치



0	5	4	-2
5	4	-2	4
4	-2	4	5
-2	4	5	0

i=2 일 때, 최적 가치



i=2 일 때, 정책

Monte-Carlo Method



■ 몬테-카를로 근사 활용

$$E[f(X)] = \int f(x)p(x)dx \approx \frac{1}{S} \sum_{i=1}^S f(x_i)$$

$$v_{\pi}(s) = \sum_{\text{s에서 출발하는 모든 경로 } z} P(z)r(z)$$

G_t : 현 시점에서 기대되는 누적 보상
 $r(z)$: 에피소드의 진행 경로에 대한 누적 보상

$$v_{\pi}(s) = \frac{1}{|Z(s)|} \sum_{z \in Z(s)} r(z), \quad \forall s \in \mathcal{S}$$

$$q_{\pi}(s, a) = \frac{1}{|Z(s, a)|} \sum_{z \in Z(s, a)} r(z), \quad \forall s \in \mathcal{S}, \quad \forall a \in \mathcal{A}$$

$Z(s)$: 수집된 상태 샘플
 $Z(s, a)$: 수집된 행동 샘플

$$v^{(t+1)}(s) = \max_{a \in \mathcal{A}(s)} \sum_{s'} \sum_r \boxed{P(s', r | s, a)} (r + v^{(t)}(s')), \quad \forall s \in \mathcal{S}$$

최적의 $q_{\pi}(s, a)$ 를 찾으면, MDP의 Dynamics를 몰라도 된다!

=> 에피소드를 수집할 수 있다면, 환경모델이 없어도 된다!!

■ 에피소드로부터 샘플 집합 구축

- 에피소드 표기

$$e = [s_0, r_0]a_0[s_1, r_1]a_1[s_2, r_2]a_2 \cdots [s_t, r_t]a_t \cdots [s_T, r_T] \quad (9.20)$$

- 예시, $e = [10,0]$ 동 $[11,0]$ 북 $[7,0]$ 남 $[11,0]$ 서 $[10,0]$ 서 $[9,0]$ 북 $[5,0]$ 북 $[1,5]$

- 상태 샘플 수집 예

$$\begin{cases} Z(5) = \{[5,0]북[1,5]\} \\ Z(7) = \{[7,0]남[11,0]서[10,0]서[9,0]북[5,0]북[1,5]\} \\ Z(9) = \{[9,0]북[5,0]북[1,5]\} \\ Z(10) = \{[10,0]동[11,0]북[7,0]남[11,0]서[10,0]서[9,0]북[5,0]북[1,5], [10,0]서[9,0]북[5,0]북[1,5]\} \\ Z(11) = \{[11,0]북[7,0]남[11,0]서[10,0]서[9,0]북[5,0]북[1,5], [11,0]서[10,0]서[9,0]북[5,0]북[1,5]\} \end{cases}$$

- 행동 샘플 수집 예

$$\begin{cases} Z(5, 북) = \{[5,0]북[1,5]\} \\ Z(7, 남) = \{[7,0]남[11,0]서[10,0]서[9,0]북[5,0]북[1,5]\} \\ Z(9, 북) = \{[9,0]북[5,0]북[1,5]\} \\ Z(10, 동) = \{[10,0]동[11,0]북[7,0]남[11,0]서[10,0]서[9,0]북[5,0]북[1,5]\} \\ Z(10, 서) = \{[10,0]서[9,0]북[5,0]북[1,5]\} \\ Z(11, 북) = \{[11,0]북[7,0]남[11,0]서[10,0]서[9,0]북[5,0]북[1,5]\} \\ Z(11, 서) = \{[11,0]서[10,0]서[9,0]북[5,0]북[1,5]\} \end{cases}$$

에피소드 끝까지 샘플 획득

■ 정책 평가

알고리즘 9-4 정책 평가를 위한 몬테카를로 방법(상태-행동 가치함수 버전)

입력: 정책 π , 에피소드 발생기

출력: 가치함수 q_π

```
1 for ( $s \in \mathcal{S}$ 와  $a \in \mathcal{A}$ )  $q_\pi(s, a) = 0$  // 가치함수를 0으로 초기화
2 repeat
3      $\pi$ 에 따라, 에피소드  $e$ 를 생성한다. <= 시뮬레이션 혹은 사전에 수집된 에피소드를 활용
4      $e$ 에서 상태-행동 샘플을 수집하여  $Z(s, a)$ 에 추가한다.
5     for (라인 4에서  $Z(s, a)$ 가 변경된  $s, a$  쌍에 대해)
6          $q_\pi(s, a) = \frac{1}{|Z(s, a)|} \sum_{z \in Z(s, a)} \mathbb{r}(z)$  // 식 (9.22)
7 until(멈춤 조건)
```

■ 최적 정책 학습

알고리즘 9-6 최적 정책 탐색을 위한 몬테카를로 방법(ϵ -소프트)

입력: 에피소드 발생기, 확률 배분 비율 ϵ

출력: 최적 정책 $\hat{\pi}$, 최적 상태-행동 가치함수 $\hat{q}$

```
1 for( $s \in \mathcal{S}$  and  $a \in \mathcal{A}$ )  $Z(s, a) = \emptyset$ ,  $q(s, a) = 0$  // 샘플집합을 공집합, 가치함수를 0 초기화
2 정책  $\pi$ 를 임의로 초기화한다.
3 repeat
4      $\pi$ 에 따라 에피소드  $e$ 를 생성한다.
5      $e$ 에서 샘플을 수집하여 훈련집합  $Z(s, a)$ 에 추가한다.
6     for(라인 5에서  $Z(s, a)$ 가 변한  $s$ ,  $a$  쌍에 대해) // 정책 평가
7          $q(s, a) = \frac{1}{|Z(s, a)|} \sum_{z \in Z(s, a)} \mathbb{I}(z)$  // 식 (9.22)
8     for( $e$ 에 있는  $s$ 에 대해)
9          $a' = \arg \max_a q(s, a)$ 
10         $A(s) = a'$ 
11        for ( $a \in \mathcal{A}(s)$ )
12             $P(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|}, & a = a' \text{ 이면} \\ \frac{\epsilon}{|\mathcal{A}(s)|}, & a \neq a' \text{ 이면} \end{cases}$  //  $\epsilon$ -소프트
13 until(멈춤 조건)
14  $\hat{q} = q$ 
15 for ( $s \in \mathcal{S}$ ) //  $A(s)$  정보를 가지고 최적 정책 구축
16      $\sum_{a \in A(s)} P(a|s) = 1$  이 되도록  $A(s)$ 에 속한 행동  $a$ 에 확률을 배분한 후 확률을  $\hat{\pi}$ 에 저장한다.
```

Monte-Carlo Method 정리



$$v_{\pi}(s_t) = \frac{1}{|Z(s_t)|} \sum_{z \in Z(s_t)} r(z)$$

- 샘플 z_t 가 k 번째로 추가되었다면 다음 식이 성립
 - $v_{\pi}^{(old)}$ 는 직전까지 가치함숫값, 즉 먼저 추가된 $k-1$ 개 샘플의 평균

$$v_{\pi}^{(new)}(s_t) = \frac{v_{\pi}^{(old)}(s_t) * (k - 1)}{k} + \frac{r(z_t)}{k} = v_{\pi}^{(old)}(s_t) + \boxed{\frac{1}{k}} \left(r(z_t) - v_{\pi}^{(old)}(s_t) \right)$$

$$v_{\pi}(s_t) = v_{\pi}(s_t) + \boxed{\rho} \left(r_{new} - v_{\pi}(s_t) \right)$$

정책을 갱신하기 위해서는 하나의 에피소드가 필요하다.

=> State가 바뀔 때 마다(1 step 마다) 갱신할 수 있는 방법은 없을까?

$$v_{\pi}(s_t) = v_{\pi}(s_t) + \rho(r_{new} - v_{\pi}(s_t))$$

가치 반복 알고리즘에서의 순환 관계 적용

$$v_{\pi}(s_t) = v_{\pi}(s_t) + \rho \left(\underbrace{(r_{t+1} + \gamma v_{\pi}(s_{t+1}))}_{TD \text{ Target}} - v_{\pi}(s_t) \right)$$

$$q_{\pi}(s_t, a_t) = q_{\pi}(s_t, a_t) + \rho \left(\underbrace{(r_{t+1} + \gamma q_{\pi}(s_{t+1}, a_{t+1}))}_{TD \text{ Target}} - q_{\pi}(s_t, a_t) \right)$$

알고리즘 9-7 정책 평가를 위한 시간차 학습(상태 가치함수 버전)

입력: 정책 π , 에피소드 발생기

출력: 가치함수 v_{π}

```
1 for ( $s \in \mathcal{S}$ )  $v_{\pi}(s) = 0$  // 가치함수를 0으로 초기화
2 repeat
3     에피소드  $e$ 를 생성한다. // 식 (9.20)
4     for ( $t=0$  to  $T-1$ )
5          $v_{\pi}(s_t) = v_{\pi}(s_t) + \rho(r_{t+1} + \gamma v_{\pi}(s_{t+1}) - v_{\pi}(s_t))$  // 식 (9.24)
6 until(멈춤 조건)
```

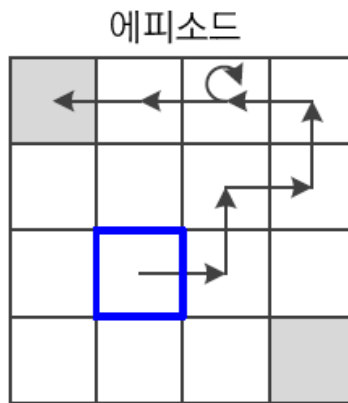

TD Learning (예제)



$e = [10,0]$ 동 $[11, -1]$ 북 $[7, -1]$ 동 $[8, -1]$ 북 $[4, -1]$ 서 $[3, -1]$ 북 $[3, -1]$ 서 $[2, -1]$ 서 $[1,5]$

정책 π

	↕	↕	↕
↔	↕	↕	↕
↔	↕	↕	↕
↔	↕	↕	



초기 가치함수

	0	0	0
0	0	0	0
0	0	0	0
0	0	0	

	-0.1	-0.2	-0.1
0	0	-0.1	-0.1
0	-0.1	-0.1	0
0	0	0	

(a) 정책과 예제 에피소드

(b) 가치함수 초기화

(c) 시간차 적응

라인 4에서 $t = 0$ 일 때 라인 5를 수행하면 다음과 같다.

$$v_{\pi}(10) = v_{\pi}(10) + 0.1(-1 + 1 * v_{\pi}(11) - v_{\pi}(10)) = -0.1$$

$t = 1$ 일 때 라인 5를 수행하면 다음과 같다.

$$v_{\pi}(11) = v_{\pi}(11) + 0.1(-1 + 1 * v_{\pi}(7) - v_{\pi}(11)) = -0.1$$

TD의 최적 정책 찾기 1 : SARSA



■ TD에서의 최적 정책 찾기 1: SARSA 알고리즘

알고리즘 9-8 최적 정책을 탐색하는 Sarsa

입력: 에피소드 발생기

출력: 최적 상태-행동 가치함수 $\hat{q}$

```
1 for( $s \in \mathcal{S}$  and  $a \in \mathcal{A}$ ) 임의값으로  $q(s, a)$ 를 설정한다.  $s$ 가 종료 상태이면  $q(s, a) = 0$ 
2 repeat
3     초기 상태  $s$ 를 생성한다.
4      $q$ 에 따라, 상태  $s$ 에서 행동  $a$ 를 생성한다. 필요하다면  $\epsilon$ -소프트를 적용한다.
5     repeat
6          $a$ 를 실행하여 다음 상태  $s'$ 와 보상  $r$ 을 얻는다. //  $q$ 가 MDP의 역할 대행함
7          $q$ 에 따라, 상태  $s'$ 에서 행동  $a'$ 를 생성한다. 필요하다면  $\epsilon$ -소프트를 적용한다.
8          $q(s, a) = q(s, a) + \rho(r + \gamma q(s', a') - q(s, a))$  // 식 (9.25)
9          $s = s', a = a'$ 
10    until( $s$ 가 종료 상태)
11 until(멈춤 조건)
12  $\hat{q} = q$ 
```

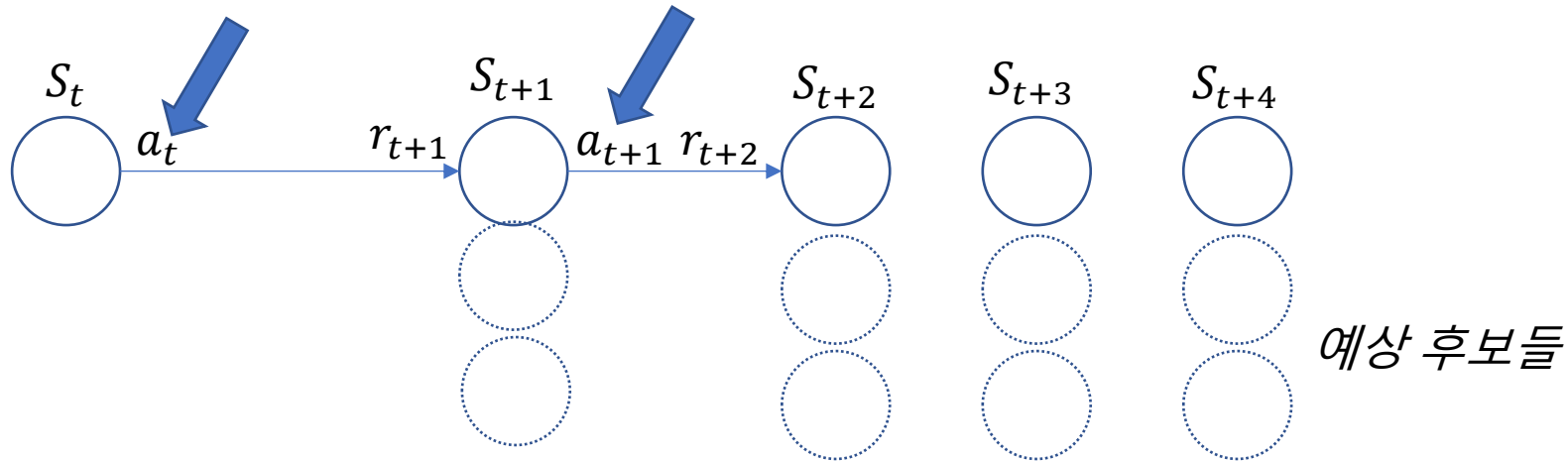
On-Policy vs Off-Policy



$$q_{\pi}(s_t, a_t) = q_{\pi}(s_t, a_t) + \rho \left((r_{t+1} + \gamma \boxed{q_{\pi}(s_{t+1}, a_{t+1})}) - q_{\pi}(s_t, a_t) \right)$$

현 시점에서 선택되는 액션을
결정하는 정책: **Behavior Policy**

미래 시점에서 선택되는 액션을
결정하는 정책: **Target Policy**



- **On-Policy:** Behavior Policy = Target Policy
- **Off-Policy:** Behavior Policy $\neq$ Target Policy

SARSA 는 On-policy

Off-Policy의 장점은 무엇일까?

TD의 최적 정책 찾기 2 : Q-Learning



■ TD에서의 최적 정책 찾기 2: Q-Learning => Off-policy

$$q_{\pi}(s_t, a_t) = q_{\pi}(s_t, a_t) + \rho ((r_{t+1} + \gamma \max_a q_{\pi}(s_{t+1}, a)) - q_{\pi}(s_t, a_t))$$

SARSA와 다른 점: $q_{\pi}(s_{t+1}, a_{t+1})$ 이 $\max_a q_{\pi}(s_{t+1}, a)$ 로 바뀐 것

즉, Behavior Policy와 Target Policy가 다르다.

Behavior Policy는 Exploring을 지향하고,

Target Policy는 Exploiting을 지향하게 하여,

적극적으로 탐험하면서 빠르게 Optimal Policy로 수렴 가능

절벽 예)

start	R=-10	R=-10	R=10

**Q-Learning의 최대 단점은
무엇일까?**

Deep Q Network (DQN)

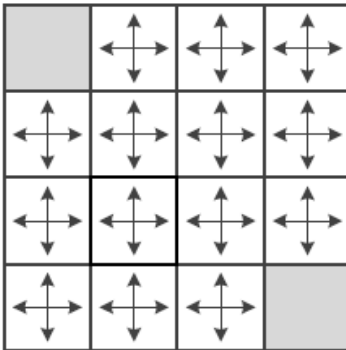


지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

■ DQN: Neural Network + Q-Learning

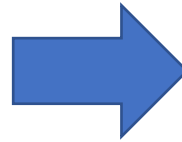
$$q_{\pi}(s_t, a_t) = q_{\pi}(s_t, a_t) + \rho ((r_{t+1} + \gamma \max_a q_{\pi}(s_{t+1}, a)) - q_{\pi}(s_t, a_t))$$

Grid world 환경



$q_{\pi}(s_t, a_t)$

Table 형태로 저장



$q_{\pi}(s_t, a_t)$

Table 형태로 저장 불가

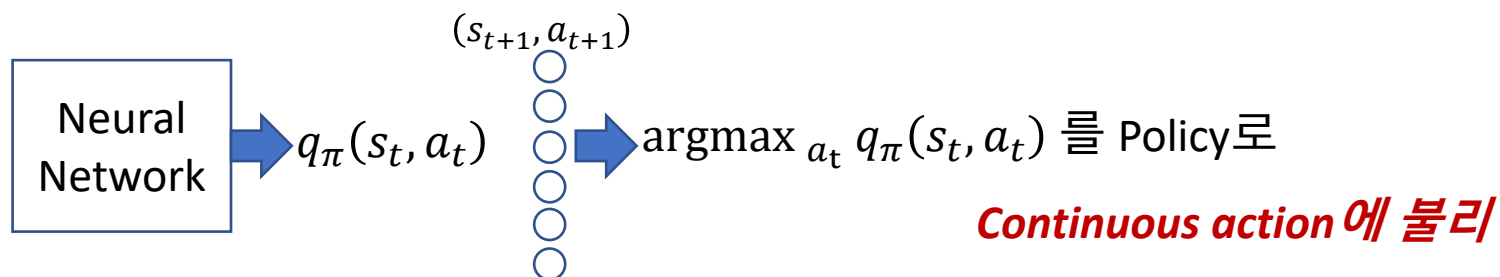
$$q_{\pi}(s_t, a_t) \Rightarrow q_{\pi}(s_t, a_t | \theta_t)$$

Key Idea: 가치 함수 q 를 신경망으로 구현

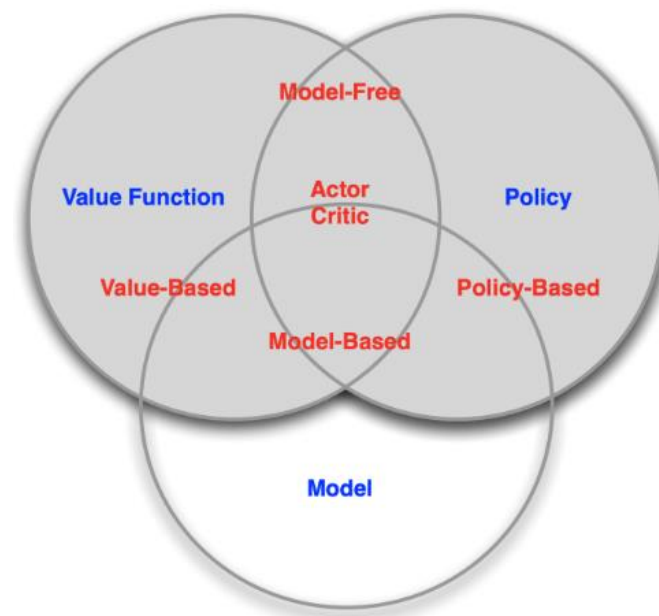
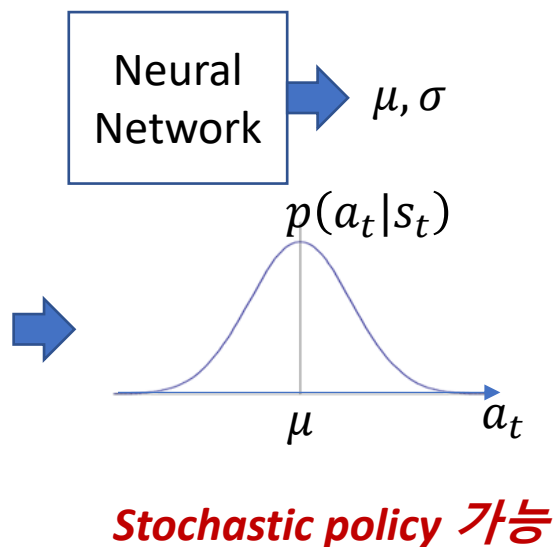
Value-Based vs Policy-Based



- Value-Based => Value-function을 optimize



- Policy-Based => $p(a_t|s_t)$ 자체를 optimize

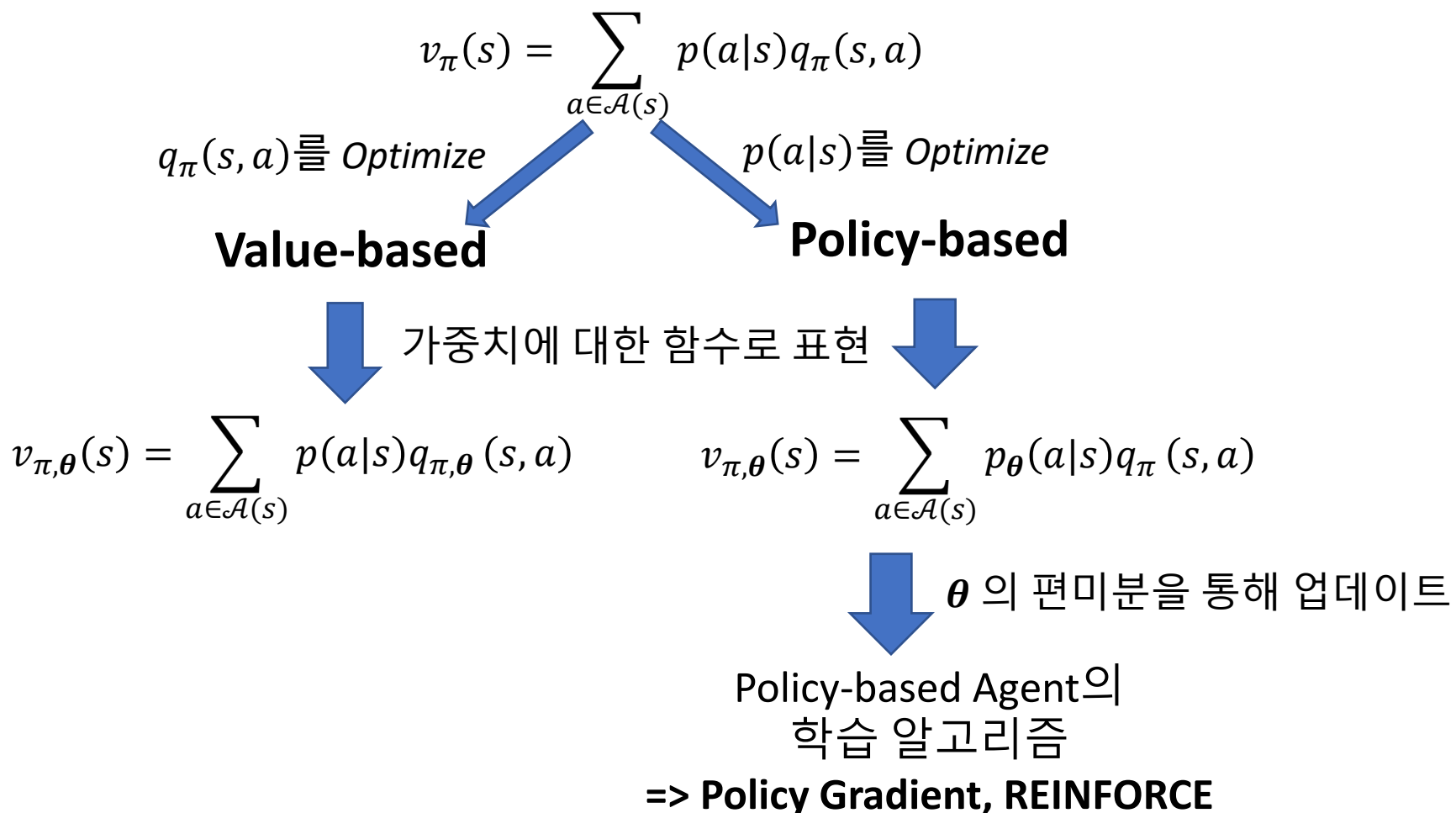


Agent의 종류

Value-Based vs Policy-Based



■ Deep Reinforcement Learning 관점



[심화] Policy Gradient



■ Continuous Action으로 접근

$$G_0 = R_0 + \gamma R_1 + \gamma^2 R_2 + \dots \text{일 때,}$$

Objective Function $J = E[G_0] = \int G_0 P(s_0, a_0, s_1, a_1, s_2, a_2 \dots) ds_0, a_0, s_1, a_1, s_2, a_2 \dots$
 $s_0, a_0, s_1, a_1, s_2, a_2 \dots$

$$\tau = s_0, a_0, s_1, a_1, s_2, a_2 \dots$$

$$J = \int_{\tau} G_0 P(\tau) d\tau \quad \xrightarrow{\text{p 함수를}} \quad J_{\theta} = \int_{\tau} G_0 P_{\theta}(\tau) d\tau$$

인공신경망으로 나타내기 위해

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} J(\theta)$$

$$\nabla_{\theta} J_{\theta} = \nabla_{\theta} \int_{\tau} G_0 P_{\theta}(\tau) d\tau = \int_{\tau} G_0 \nabla_{\theta} P_{\theta}(\tau) d\tau$$

$$\nabla_{\theta} P_{\theta}(\tau) = P_{\theta}(\tau) \nabla_{\theta} \ln(P_{\theta}(\tau))$$

$$E[f(X)] = \int f(x)p(x)dx \approx \frac{1}{S} \sum_{i=1}^S f(x_i)$$

$$\nabla_{\theta} J_{\theta} = \int_{\tau} G_0 (\nabla_{\theta} \ln(P_{\theta}(\tau))) P_{\theta}(\tau) d\tau$$

[심화] Policy Gradient



$$\nabla_{\theta} J_{\theta} = \int_{\tau} G_0 (\nabla_{\theta} \ln(P_{\theta}(\tau))) P_{\theta}(\tau) d\tau$$

$$\begin{aligned} P_{\theta}(\tau) &= P_{\theta}(s_0, a_0, s_1, a_1, s_2, a_2 \dots) \\ &= P_{\theta}(s_0) P_{\theta}(a_0, s_1, a_1, s_2, a_2 \dots | s_0) \\ &= P_{\theta}(s_0) P_{\theta}(a_0 | s_0) P_{\theta}(s_1, a_1, s_2, a_2 \dots | s_0, a_0) \\ &= P_{\theta}(s_0) P_{\theta}(a_0 | s_0) P_{\theta}(s_1, a_1, s_2, a_2 \dots | s_0, a_0) \\ &= P_{\theta}(s_0) P_{\theta}(a_0 | s_0) P_{\theta}(s_1 | s_0, a_0) P_{\theta}(a_1, s_2, a_2 \dots | s_0, a_0, s_1) \\ &= P_{\theta}(s_0) P_{\theta}(a_0 | s_0) P_{\theta}(s_1 | s_0, a_0) P_{\theta}(a_1 | s_0, a_0, s_1) P_{\theta}(s_2, a_2 \dots | s_0, a_0, s_1, a_1) \end{aligned}$$

계속 전개 가능

$$P(A, B) = P(A|B)P(B) = P(B|A)P(A)$$

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

$$P(A, B|C) = P(A|C)P(B|A, C)$$

[심화] Policy Gradient



$$\nabla_{\theta} J_{\theta} = \int_{\tau} G_0 (\nabla_{\theta} \ln(P_{\theta}(\tau))) P_{\theta}(\tau) d\tau$$

$$\begin{aligned} P_{\theta}(\tau) &= P_{\theta}(s_0, a_0, s_1, a_1, s_2, a_2 \dots) \\ &= P_{\theta}(s_0) P_{\theta}(a_0, s_1, a_1, s_2, a_2 \dots | s_0) \\ &= P_{\theta}(s_0) P_{\theta}(a_0 | s_0) P_{\theta}(s_1, a_1, s_2, a_2 \dots | s_0, a_0) \\ &= P_{\theta}(s_0) P_{\theta}(a_0 | s_0) P_{\theta}(s_1, a_1, s_2, a_2 \dots | s_0, a_0) \\ &= P_{\theta}(s_0) P_{\theta}(a_0 | s_0) P_{\theta}(s_1 | s_0, a_0) P_{\theta}(a_1, s_2, a_2 \dots | s_0, a_0, s_1) \\ &= P_{\theta}(s_0) P_{\theta}(a_0 | s_0) P_{\theta}(s_1 | s_0, a_0) P_{\theta}(a_1 | s_0, a_0, s_1) P_{\theta}(s_2, a_2 \dots | s_0, a_0, s_1, a_1) \end{aligned}$$

$$P(A, B) = P(A|B)P(B) = P(B|A)P(A)$$

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

$$P(A, B|C) = P(A|C)P(B|A, C)$$

지을 수 있는 이유는?

$$P_{\theta}(\tau) = \prod P_{\theta}(a_t | s_t)$$

$$\nabla_{\theta} \ln(P_{\theta}(\tau)) = \sum \ln(P_{\theta}(a_t | s_t))$$

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} J(\theta)$$

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} v_{\pi_{\theta}}(s_0)$$

열심히 정리를 하면!

$$\approx \theta_t + \alpha [\nabla_{\theta} \log \pi_{\theta}(a|s) \boxed{q_{\pi}(s, a)}] \quad \text{q를 구할 수 없으므로}$$

$$\approx \theta_t + \alpha [\nabla_{\theta} \log \pi_{\theta}(a|s) \boxed{G_t}]$$

에피소드 끝까지의 누적보상으로 대체

꼭 누적보상으로 대체할 필요가 있을까?

$q_{\pi, \theta}(s, a)$ 를 활용하면?



Actor-Critic 알고리즘

- Actor: $p_{\theta}(a|s) \Rightarrow$ 정책 함수를 Neural Network 로
- Critic: $q_{\pi, \theta}(s, a) \Rightarrow$ 가치 함수를 Neural Network 로

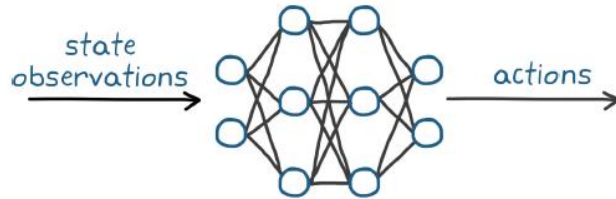
Actor-Critic



Policy-based

$$v_{\pi, \theta}(s) = \sum_{a \in \mathcal{A}(s)} p_{\theta}(a|s) q_{\pi}(s, a)$$

State에 대한 함수



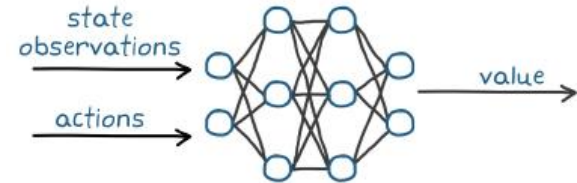
ACTOR

정책 Network를 업데이트

Value-based

$$v_{\pi, \theta}(s) = \sum_{a \in \mathcal{A}(s)} p(a|s) q_{\pi, \theta}(s, a)$$

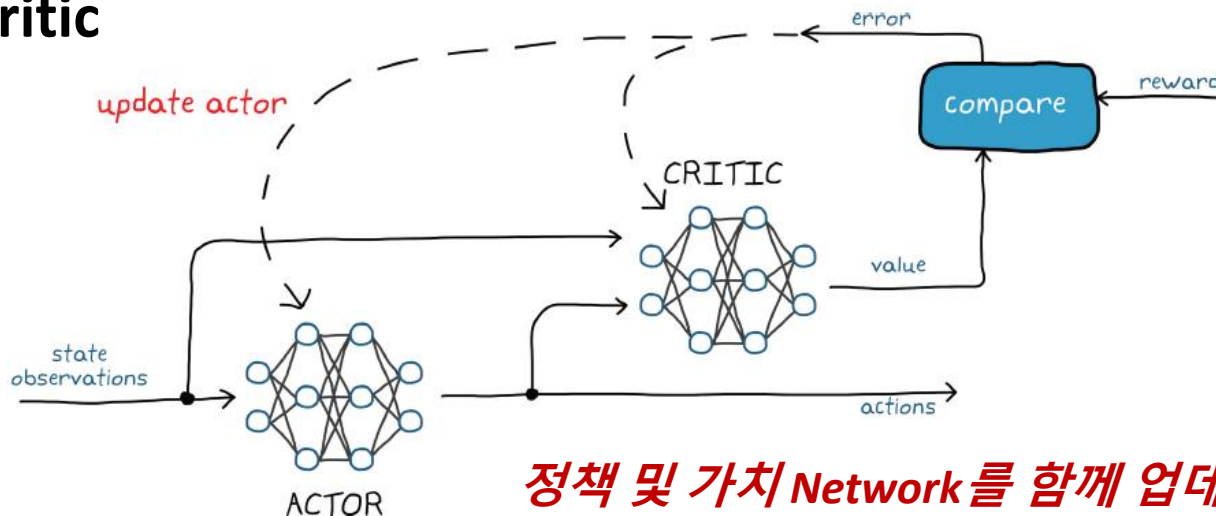
State와 Action에 대한 함수



CRITIC

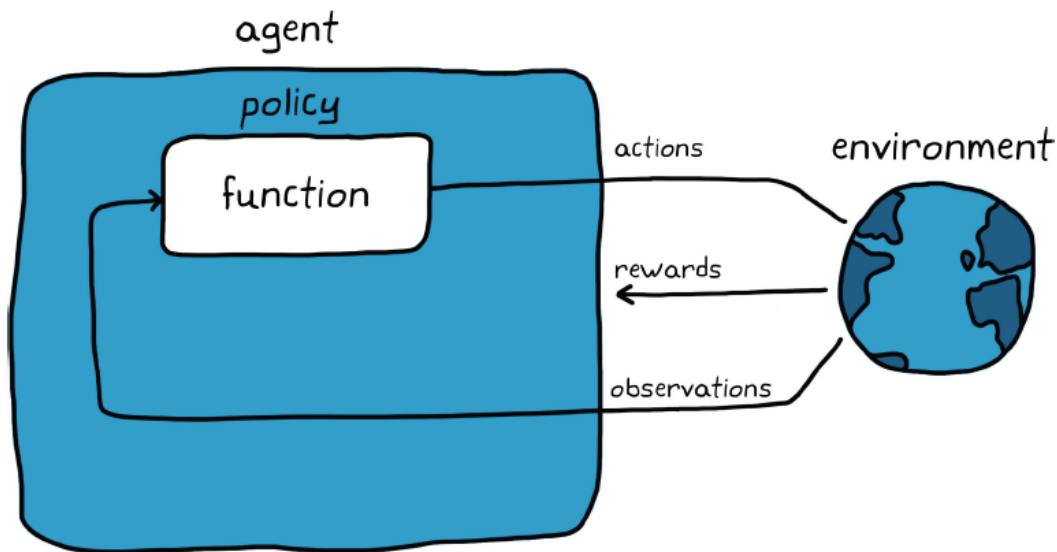
가치 Network를 업데이트

Actor-Critic



정책 및 가치 Network를 함께 업데이트

지금까지는 Agent(policy)에 대한 이론이었습니다.



■ Simulation or 실제 Testbed

■ Observation 결정

- 환경에서 얻을 수 있는 관측 값 (Scalar)
- 예) 센서 데이터 등
- 일반적으로, Continuous space로 구현

■ Action 결정

- 현 state에서 제어할 움직임 값 (Scalar)
- 예) 모터에 가하는 전압, 전류 등
- Continuous space도 가능하나, Discrete space가 성공확률이 높음.

■ Reward

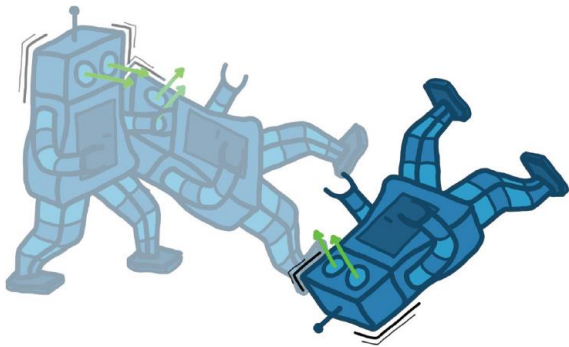
- 스칼라 값으로 결정해야 함.
- Observation으로 결정한 파라미터들을 활용해도 무방
- 알고리즘 혹은 Neural Network(Regression)로도 구현 가능

■ Episode 결정

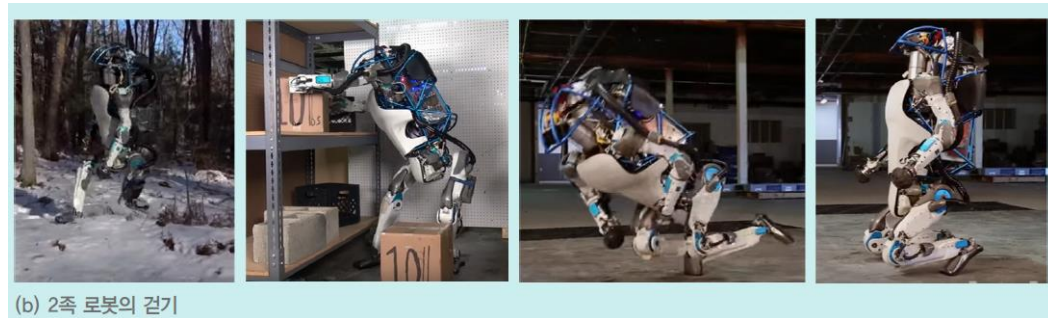
- 초기 상태를 어떻게 설정할 것인가?
- Episode를 어떻게 종료할 것인가?

■ 세부 Training

- 시뮬레이션 환경에서 Training된 agent를 실제 환경에 적용
- Agent는 실제 환경에서 추가적으로 training 되며 optimal policy를 찾음.



Simulation 환경



(b) 2족 로봇의 걷기

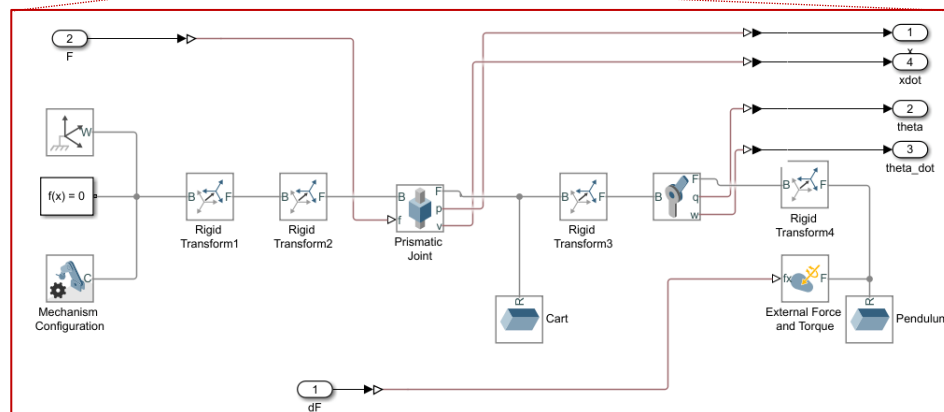
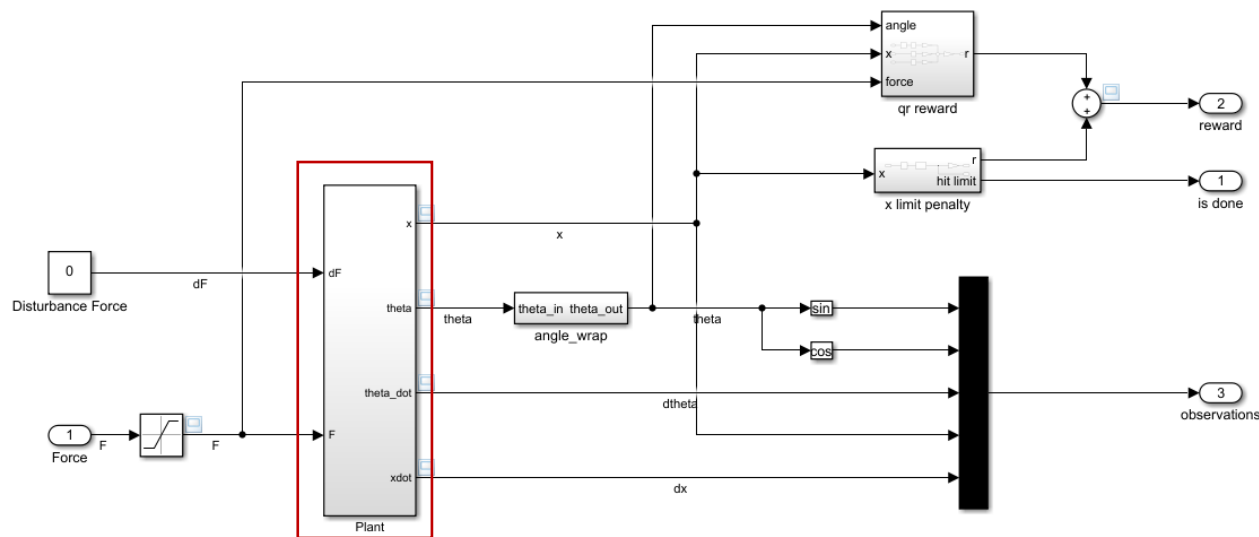
간단한 실습



<https://kr.mathworks.com/help/reinforcement-learning/ug/train-ddpg-agent-to-swing-up-and-balance-cart-pole-system.html>

카트-폴 시스템이 위쪽으로 똑바로 서서 균형을 유지하도록 DDPG 에이전트 훈련시키기

```
mdl = 'rlCartPoleSimscapeModel';  
open_system(mdl)
```



<https://kr.mathworks.com/help/reinforcement-learning/ug/train-ddpg-agent-to-swing-up-and-balance-cart-pole-system.html>

카트-폴 시스템이 위쪽으로 똑바로 서서 균형을 유지하도록 DDPG 에이전트 훈련시키기

- 위쪽으로 똑바로 균형이 잡혀 있을 때의 막대 위치는 θ 라디안이고, 아래쪽으로 매달려 있을 때의 위치는 π 라디안입니다.
- 에이전트에서 환경으로 전달되는 힘 행동 신호는 -15N 에서 15N 까지입니다.
- 환경에서 관측하는 값은 카트의 위치와 속도, 막대 각의 사인과 코사인, 도함수입니다.
- 카트가 원래 위치에서 3.5m 이상 이동하면 에피소드가 종료됩니다.
- 매 시간 스텝마다 제공되는 보상 r_t 는 다음과 같습니다.

$$r_t = -0.1(5\theta_t^2 + x_t^2 + 0.05u_{t-1}^2) - 100B$$

여기서 각 요소는 다음과 같습니다.

- θ_t 는 막대의 똑바로 세워진 위치에서의 변위 각도입니다.
- x_t 는 카트의 중심 위치에서의 위치 변위입니다.
- u_{t-1} 은 이전 시간 스텝의 제어 노력입니다.
- B 는 카트가 범위를 벗어나는지를 나타내는 플래그(1 또는 0)입니다.

간단한 실습



막대에 대해 미리 정의된 환경 인터페이스를 만듭니다.

```
env = rlPredefinedEnv('CartPoleSimscapeModel-Continuous')
```



실제로는 설계자가 직접 만들어야 함.

환경 인터페이스에서 관측값과 행동 정보를 가져옵니다.

```
obsInfo = getObservationInfo(env);  
numObservations = obsInfo.Dimension(1);  
actInfo = getActionInfo(env);
```



실제로는 설계자가 정의해야 함.

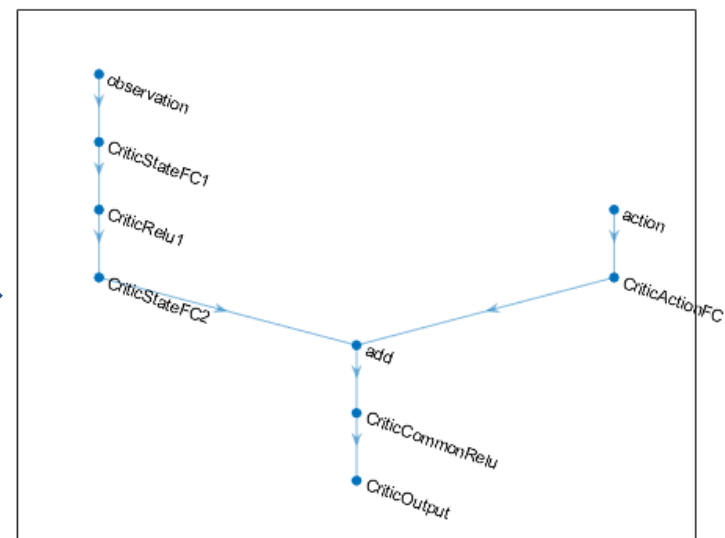
DDPG 에이전트 만들기

DDPG 에이전트는 관측값과 행동이 주어지면 크리틱 가치 함수 표현을 사용하여 장기 보상을 근사합니다. 방법에 대한 자세한 내용은 [Create Policies and Value Functions](#) 항목을 참조하십시오.

```
statePath = [  
    featureInputLayer(numObservations,'Normalization','none','Name','observation')  
    fullyConnectedLayer(128,'Name','CriticStateFC1')  
    reluLayer('Name','CriticRelu1')  
    fullyConnectedLayer(200,'Name','CriticStateFC2')];  
  
actionPath = [  
    featureInputLayer(1,'Normalization','none','Name','action')  
    fullyConnectedLayer(200,'Name','CriticActionFC1','BiasLearnRateFactor',0)];  
  
commonPath = [  
    additionLayer(2,'Name','add')  
    reluLayer('Name','CriticCommonRelu')  
    fullyConnectedLayer(1,'Name','CriticOutput')];  
  
criticNetwork = layerGraph(statePath);  
criticNetwork = addLayers(criticNetwork,actionPath);  
criticNetwork = addLayers(criticNetwork,commonPath);  
  
criticNetwork = connectLayers(criticNetwork,'CriticStateFC2','add/in1');  
criticNetwork = connectLayers(criticNetwork,'CriticActionFC1','add/in2');  
criticNetwork = dlnetwork(criticNetwork);
```



Critic 설계



Actor 설계

```
actorNetwork = [  
    featureInputLayer(numObservations,'Normalization','none','Name','observation')  
    fullyConnectedLayer(128,'Name','ActorFC1')  
    reluLayer('Name','ActorRelu1')  
    fullyConnectedLayer(200,'Name','ActorFC2')  
    reluLayer('Name','ActorRelu2')  
    fullyConnectedLayer(1,'Name','ActorFC3')  
    tanhLayer('Name','ActorTanh1')  
    scalingLayer('Name','ActorScaling','Scale',max(actInfo.UpperLimit));  
actorNetwork = dlnetwork(actorNetwork);  
  
actorOptions = rlOptimizerOptions('LearnRate',5e-04,'GradientThreshold',1);  
  
actor = rlContinuousDeterministicActor(actorNetwork,obsInfo,actInfo);
```

시뮬레이션 시간 T_f 와 에이전트 샘플 시간 T_s 를 초 단위로 지정합니다.

```
Ts = 0.02;  
Tf = 25;
```

Agent 세부 옵션

DDPG 에이전트를 만들려면 먼저 `rlDDPGAgentOptions` 객체를 사용하여 DDPG 에이전트 옵션을 지정하십시오.

```
agentOptions = rlDDPGAgentOptions(...  
    'SampleTime',Ts,...  
    'ActorOptimizerOptions',actorOptions,...  
    'CriticOptimizerOptions',criticOptions,...  
    'ExperienceBufferLength',1e6,...  
    'MiniBatchSize',128);  
agentOptions.NoiseOptions.Variance = 0.4;  
agentOptions.NoiseOptions.VarianceDecayRate = 1e-5;
```

그런 다음, 지정된 액터 표현, 크리틱 표현, 에이전트 옵션을 사용하여 에이전트를 만듭니다. 자세한 내용은 `rlDDPGAgent`

```
agent = rlDDPGAgent(actor,critic,agentOptions); Agent 최종 선언
```

간단한 실습



Agent 훈련 옵션 설정

```
maxepisodes = 2000;  
maxsteps = ceil(Tf/Ts);  
trainingOptions = rlTrainingOptions(...  
    'MaxEpisodes',maxepisodes,...  
    'MaxStepsPerEpisode',maxsteps,...  
    'ScoreAveragingWindowLength',5,...  
    'Verbose',false,...  
    'Plots','training-progress',...  
    'StopTrainingCriteria','AverageReward',...  
    'StopTrainingValue',-400,...  
    'SaveAgentCriteria','EpisodeReward',...  
    'SaveAgentValue',-400);
```

훈련 진행

```
trainingStats = train(agent,env,trainingOptions);
```

Agent 검증

DDPG 에이전트 시뮬레이션하기

훈련된 에이전트의 성능을 검증하려면 카트-폴 환경 내에서 에이전트를 시뮬레이션하십시오. 에이전트 시뮬

```
simOptions = rlSimulationOptions('MaxSteps',500);  
experience = sim(env,agent,simOptions);
```

